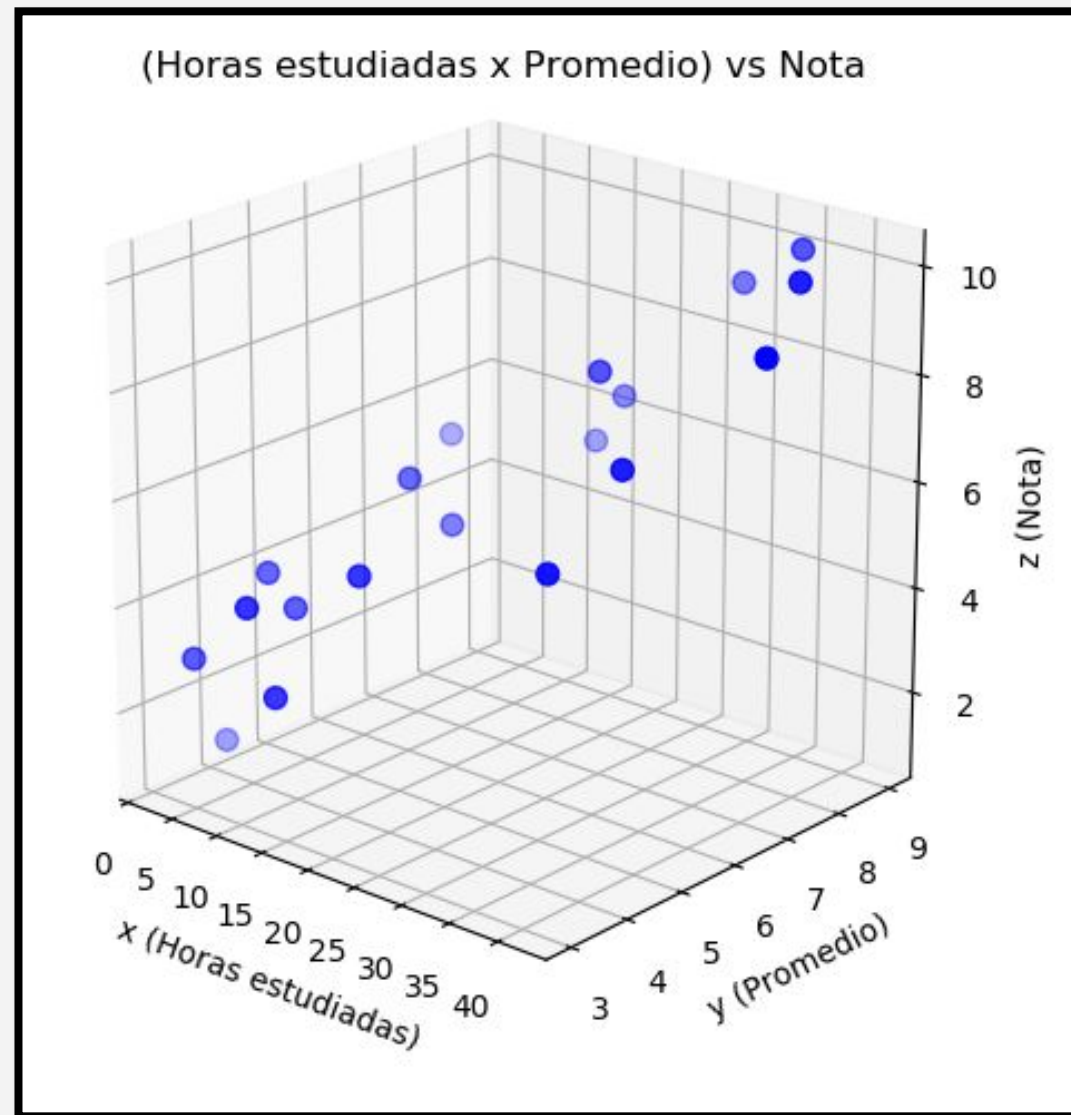


Regresión Lineal con Múltiples Variables

Datos con múltiples variables

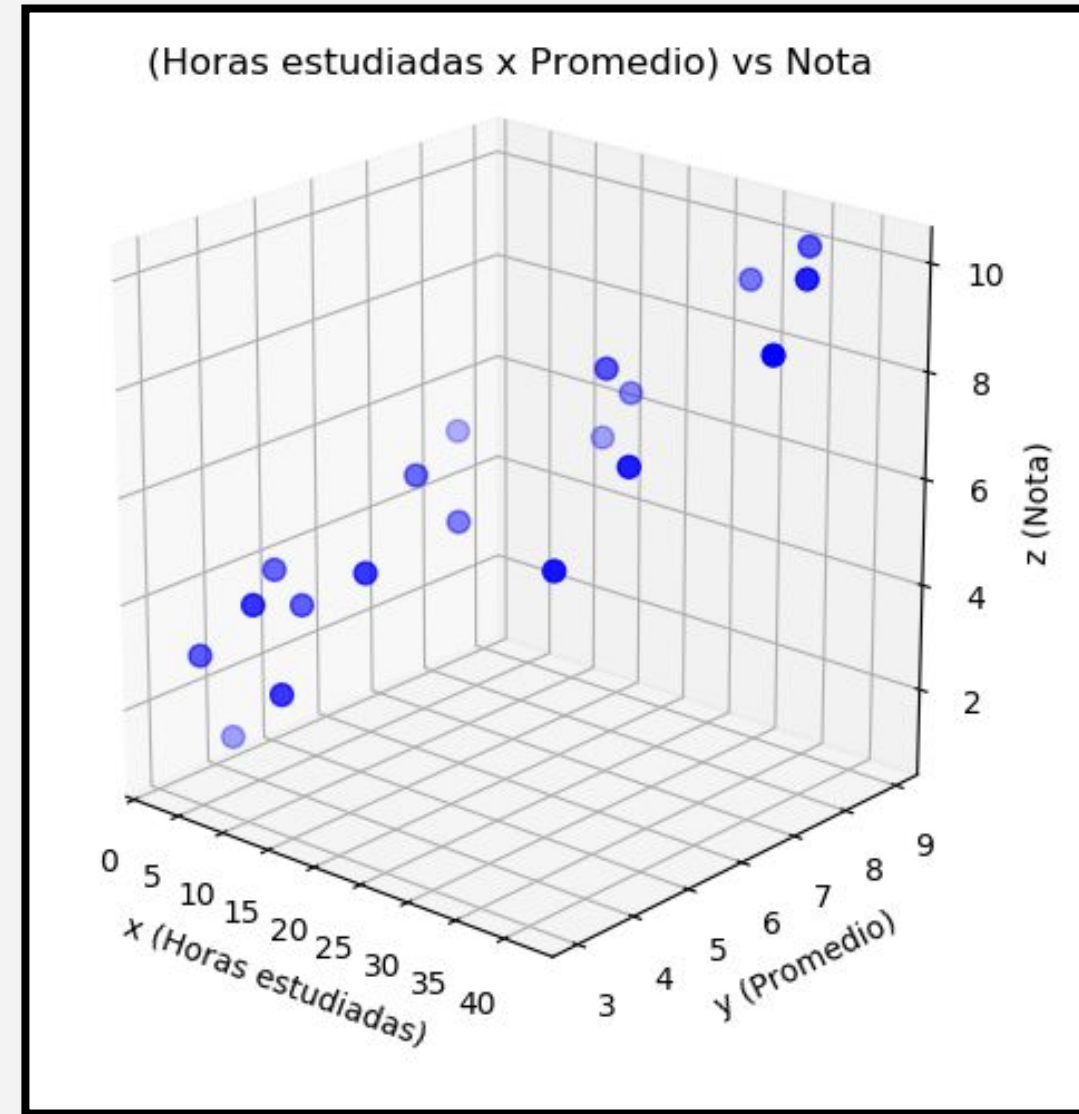
- Hasta ahora
 - Datos con una sola variable
 - 1-D
 - $x \in \mathbf{R}^1$
 - Ejemplo
 - Tiempo de estudio
- ¿Qué sucede si agregamos más información de cada ejemplo?
 - **m** variables
 - **m**-D
 - $\mathbf{x} \in \mathbf{R}^m$
 - Ejemplo
 - Horas estudiadas y promedio



Datos con 2 variables

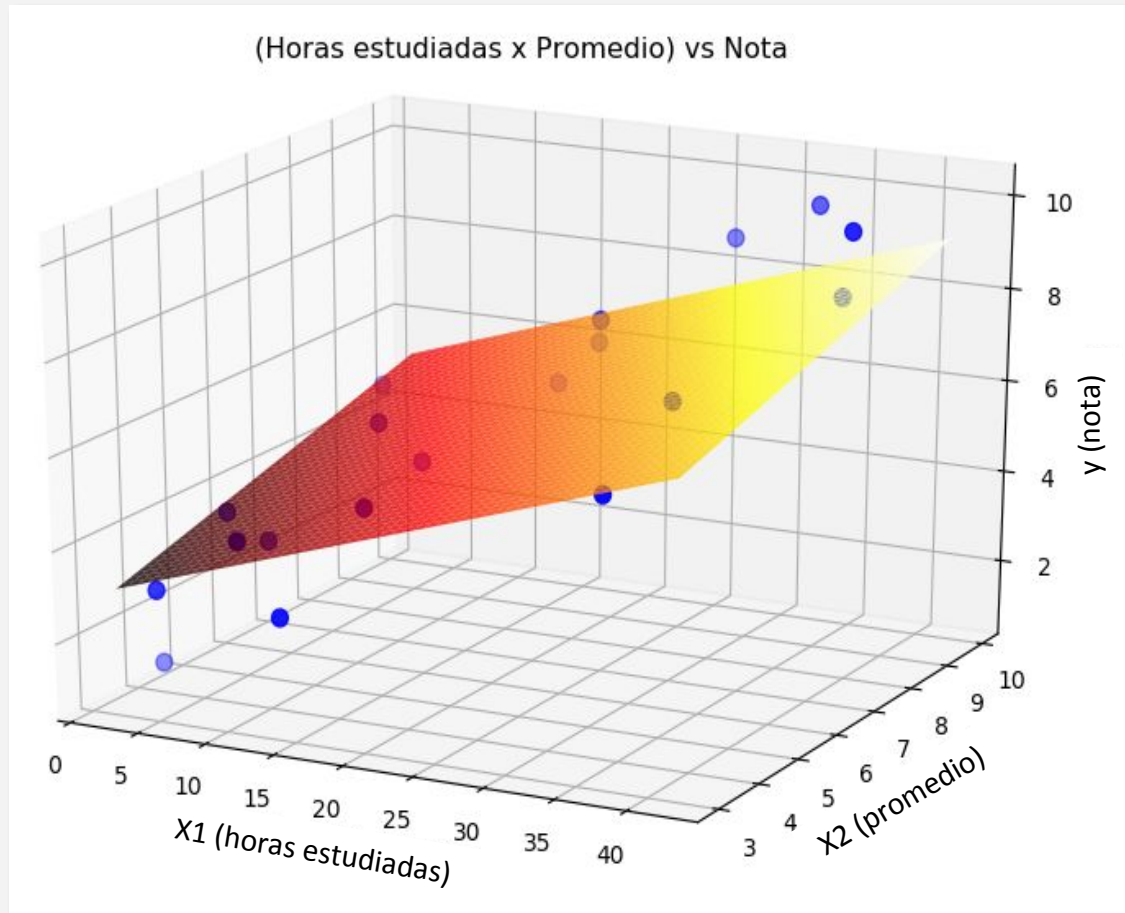
- x con subíndices
 - x_1 variable 1
 - x_2 variable 2

Estudio (x_1)	Promedio (x_2)	Nota (y)
2	4	1
5	3	3,2
7	4	4,5
9	7	6
10	4	4
11	3	4,5
13,4	5	5,5
14	3	3



Función del modelo con 2 variables

- Modelo: **plano 2D** en un **espacio 3D**
- Dos pendientes: w_1 y w_2
 - Coeficientes (o Pesos o **W**eights)



- Función del modelo
 - $f(x_1, x_2) = x_1 w_1 + x_2 w_2 + b$
- En forma vectorial
 - $\mathbf{x} = (x_1, x_2)$
 - $\mathbf{w} = (w_1, w_2)$
 - $f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} + b$
 - $\cdot$ es el **producto escalar**
 - $\mathbf{x} \cdot \mathbf{w} = x_1 w_1 + x_2 w_2$
 - $\mathbf{x} \cdot \mathbf{w} \in \mathbb{R}$

Función del modelo con 2 variables

- Ejemplo

- $\mathbf{x} = (-2, 3)$

- $\mathbf{w} = (4, 2)$

- $b = 5$

- $$\begin{aligned} f(-2,3) &= -2 w_1 + 3 w_2 + b \\ &= -2 \cdot 4 + 3 \cdot 2 + 5 \\ &= -8 + 6 + 5 \\ &= 3 \end{aligned}$$

- Forma vectorial

- $$\begin{aligned} f((-2, 3)) &= (-2, 3) \cdot \mathbf{w} + b \\ &= (-2, 3) \cdot (4, 2) + 5 \\ &= -2 + 5 \\ &= 3 \end{aligned}$$

- Los elementos de $\mathbf{w}$ son **coeficientes o pesos**

- Indican el peso relativo de cada variable

- Pesos negativos indican relación negativa

- b término constante

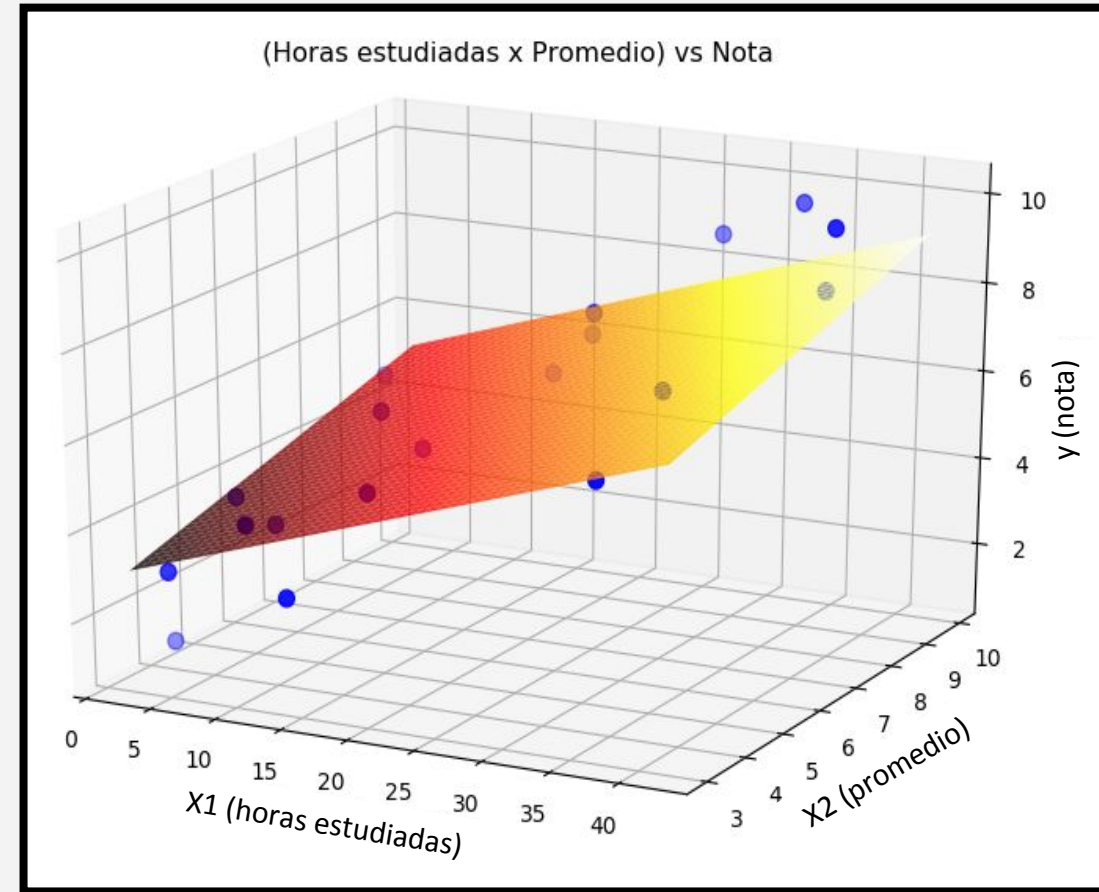
- sigue siendo la ordenada al origen en 3D

Función de error para 2 variables

- La forma de **E** se mantiene
 - Depende de 3 parámetros
 - $E(w_1, w_2, b) = 1/n \sum_i^n E_i(w_1, w_2, b)$
- La de **E_i** también
 - Pero nueva notación
 - $x_{i,j}$ = variable **j** del ejemplo **i**
 - $E_i(w_1, w_2, b) = (y_i - f(x_1, x_2))^2$
 $= (y_i - (x_{i,1} w_1 + x_{i,2} w_2 + b))^2$
 - Viendo a **x_i** y **w** como vectores
 - $E_i(\mathbf{w}, b) = (y_i - f(\mathbf{x}_i))^2$
 $= (y_i - \mathbf{x}_i \cdot \mathbf{w} + b)^2$

Optimización con **1 vs 2** variables

- ¿Qué cambia?
 - Prácticamente nada
 - Ahora tenemos **3** parámetros a optimizar
 - **w_1, w_2, b**
 - También **3** Derivadas
 - Ya no es posible graficar el error
 - Muy difícil “adivinar” los mejores parámetros probando manualmente



Derivadas de **E** con 2 variables

- Derivada parcial respecto de **w₁** o **w₂** es similar a la de **m**

$$\frac{\partial E}{\partial w_1} = \frac{2}{n} \sum_i^n (y_i - f(x_i)) x_{i,1}$$

$$\Delta E = \left(\frac{\partial E}{\partial w_1}, \frac{\partial E}{\partial w_2}, \frac{\partial E}{\partial b} \right)$$

$$\frac{\partial E}{\partial w_2} = \frac{2}{n} \sum_i^n (y_i - f(x_i)) x_{i,2}$$

$$\frac{\partial E}{\partial b} = \frac{2}{n} \sum_i^n y_i - f(x_i)$$

Función del modelo con **m** variables

- Modelo: **hiperplano** en $\mathbf{R}^m$
- **m** coeficientes: $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_m$
- Función del modelo
 - $f(x_1, x_2, \dots, x_m) = x_1 w_1 + x_2 w_2 + \dots + x_m w_m + b$
- En forma vectorial
 - $\mathbf{x} = (x_1, x_2, \dots, x_m)$
 - $\mathbf{w} = (w_1, w_2, \dots, w_m)$
 - $f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} + b$
 - $\cdot$ es el **producto escalar**
 - $\mathbf{x} \cdot \mathbf{w} = x_1 w_1 + x_2 w_2 + \dots + x_m w_m = \sum_{j=1}^n x_j w_j$
 - $\mathbf{x} \cdot \mathbf{w} \in \mathbf{R}$

Derivadas de **E** con **m** variables

- Derivada parcial respecto de **w_j** (igual para las m variables) y **b**

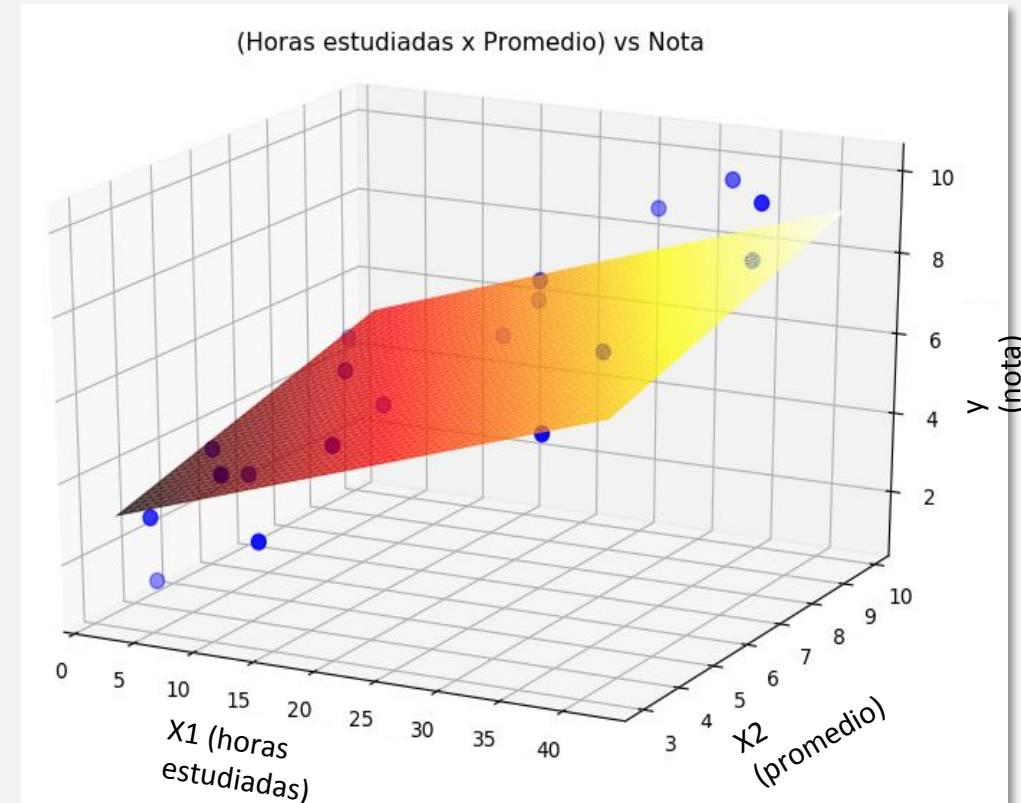
$$\Delta E = \left(\frac{\partial E}{\partial w_1}, \frac{\partial E}{\partial w_2}, \dots, \frac{\partial E}{\partial w_m}, \frac{\partial E}{\partial b} \right)$$

$$\frac{\partial E}{\partial w_j} = \frac{2}{n} \sum_i^n (y_i - f(x_i)) x_{i,j}$$

$$\frac{\partial E}{\partial b} = \frac{2}{n} \sum_i^n y_i - f(x_i)$$

Resumen

- Regresión lineal para **m** variables de entrada
 - Más información de los ejemplos
 - **m** coeficientes o pesos w_1 a w_m
 - No se puede visualizar la función de error
 - Tampoco **f** si $m > 2$
 - Derivadas similares
 - Apto para descenso de gradiente
 - Notación vectorial
 - $f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} + b$
 - Producto escalar •



1. Modificar los parámetros m_x , m_y y b
 - a. Observar como cambia el plano y el error

1. Experimentar con varios valores iniciales de los parámetros y α .

Ejercicio: Archivo **Regresión Lineal Múltiples Variables - Aprendizaje - Boston Housing.ipynb**

1. Entrenar el modelo. Intentar bajar el error lo más posible cambiando la cantidad de iteraciones, el α y los valores iniciales.
2. Luego analizar los valores de los parámetros w_i y b . ¿Qué significan?
3. Clasificar nuevos datos, cambiar los valores y observar cómo cambia el precio.
4. ¿Hay variables que no importan? ¿Cómo lo veo desde los valores del nuevo dato? ¿y desde el modelo? ¿cómo se refleja eso en las derivadas?
5. Repetir con Regresion Lineal Multiple -Aprendizaje - Vinos.ipynb.

Regresión Lineal

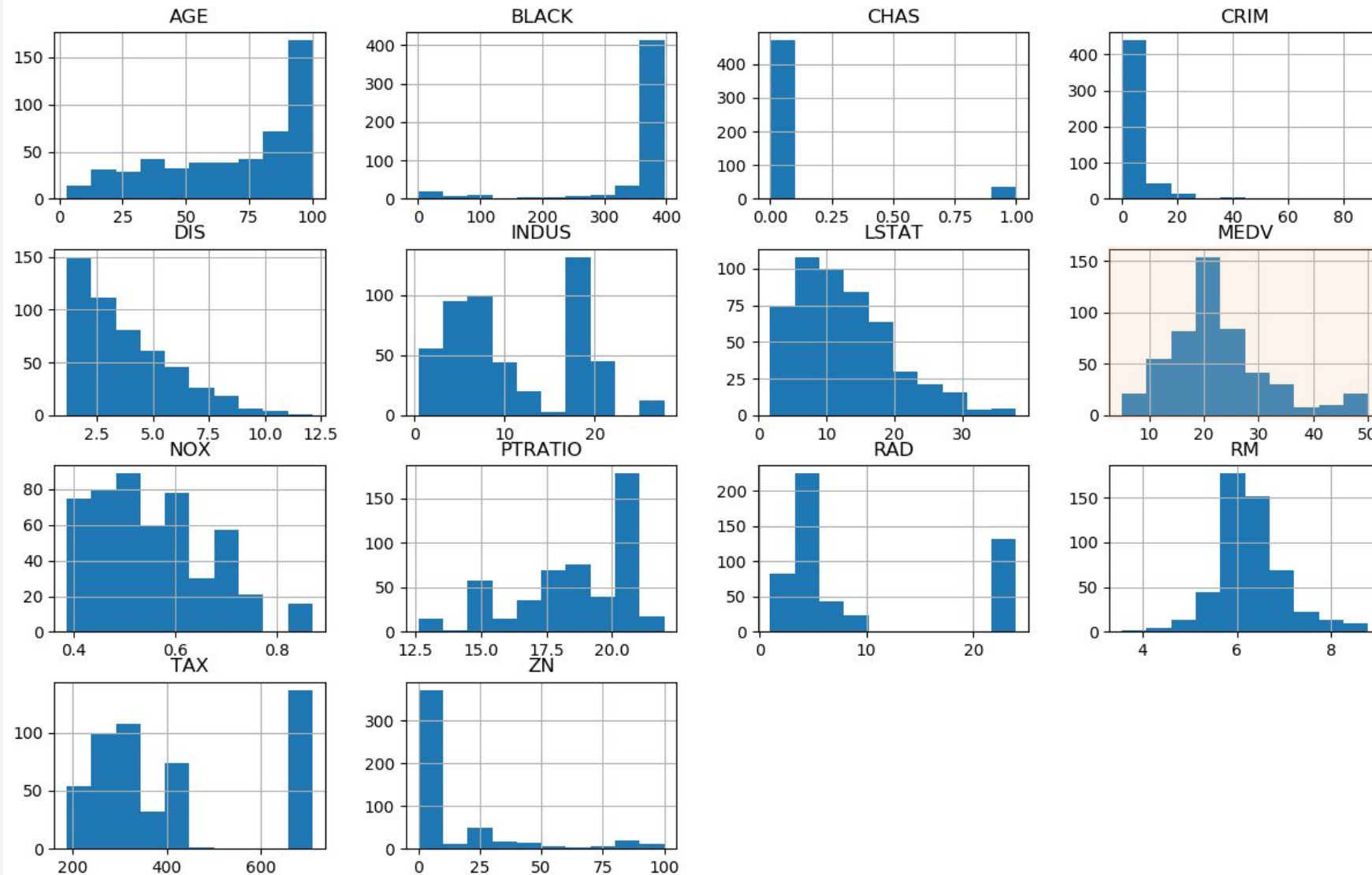
Interpretación del Modelo

Ejemplo regresión lineal. Boston Housing Dataset.

- Objetivo
- Predecir el precio de una casa
- Datos
 - De la casa
 - De la zona
- Variable **black**
 - ¿qué implica para el modelo?

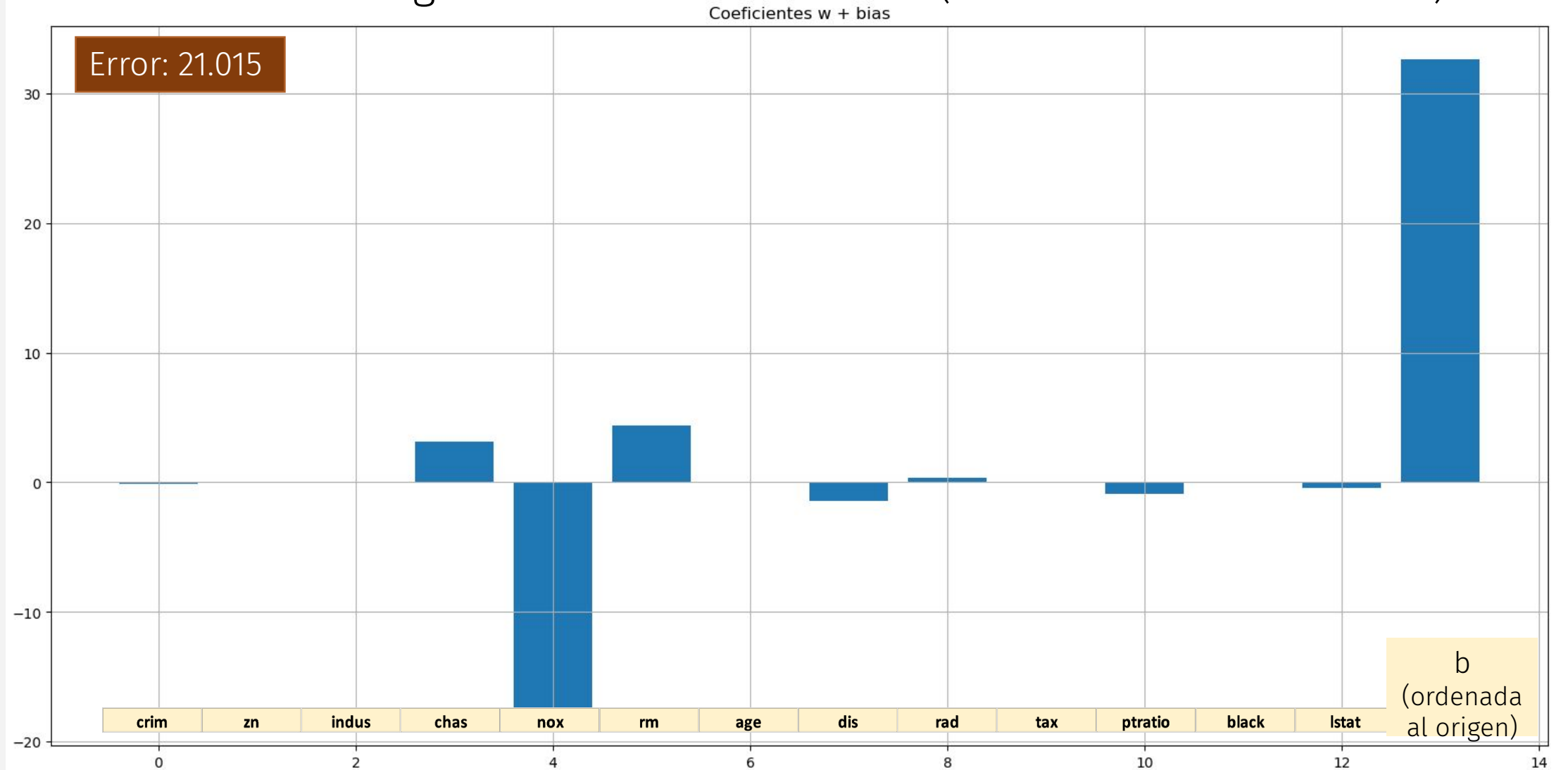
Variable	Detalle
crim	crímenes per cápita
zn	proporción de áreas residenciales
indus	proporción de negocios no retail
chas 1	si está cerca del río, 0 sino
nox	concentración de nitrógeno
rm	habitaciones promedio
age	proporción de inmuebles anteriores a 1940
dis	distancia promedio a centros de empleo
rad	accesibilidad por autopistas
tax	impuestos
ptratio	relación alumno-docente en escuelas
black	coeficiente de personas negras (dataset del 1980!)
lstat	porcentaje de personas de bajo status
Medv	Valor medio de viviendas ocupadas por sus dueños

Histogramas de valores de cada variable



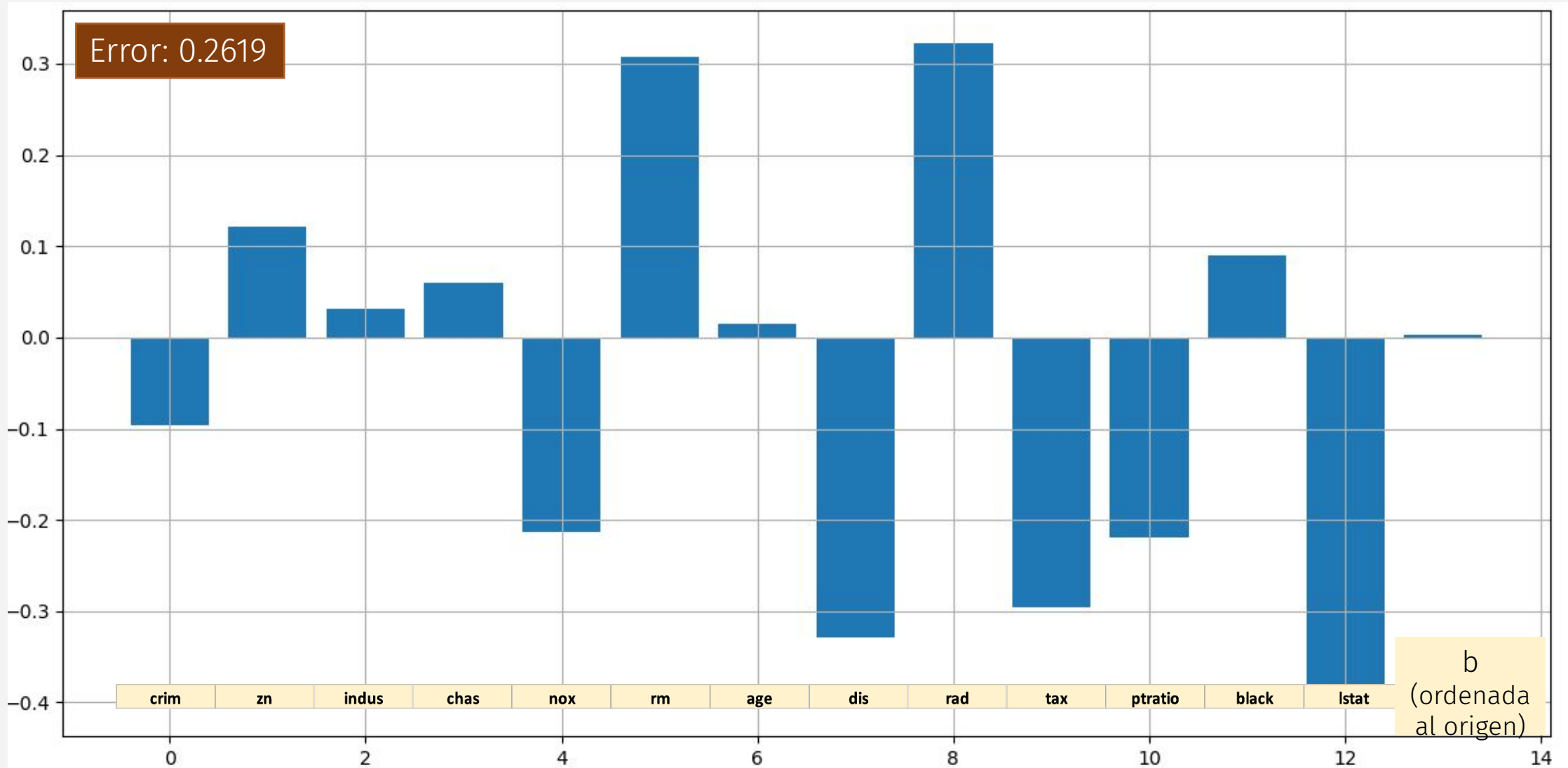
Modelo entrenado. Boston Housing Dataset.

Coeficientes luego de entrenar el modelo (variables sin normalizar)



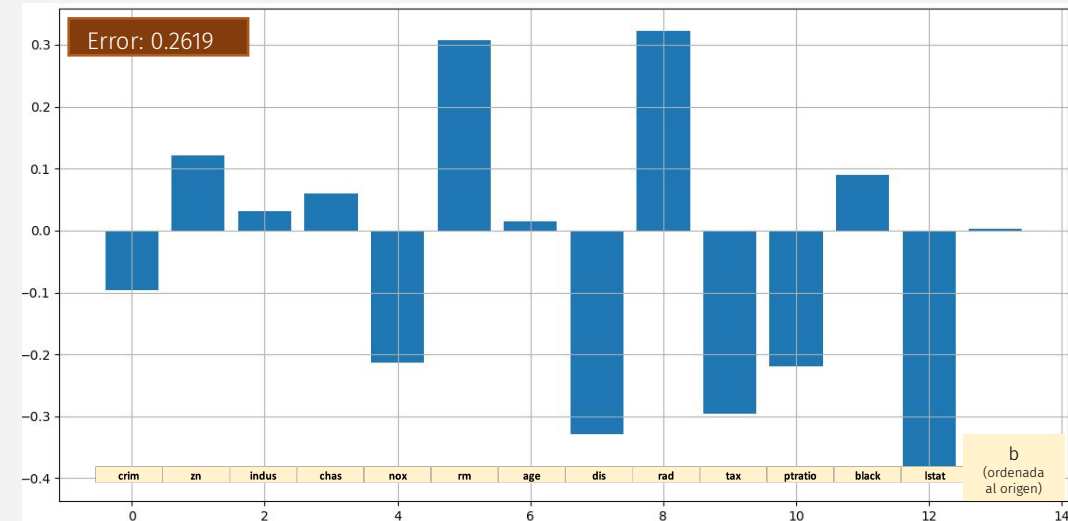
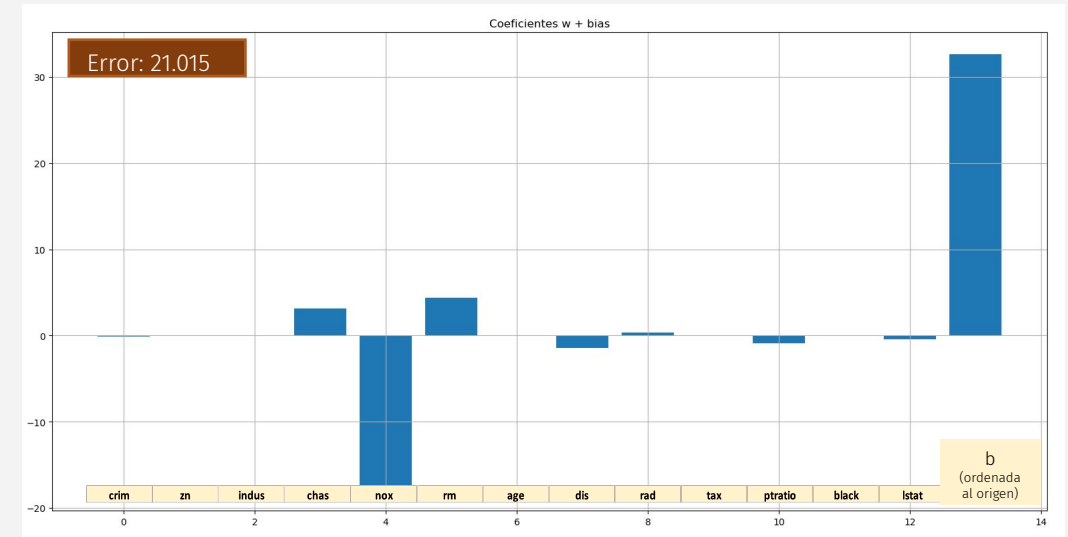
Modelo entrenado. Boston Housing Dataset.

Coeficientes luego de entrenar el modelo (con variables normalizadas)



Resumen

- Regresión Lineal
 - Podemos interpretar coeficientes
 - Modelo de “caja blanca”
 - **Escala** de **coeficientes** *depende* de **escala** de **variables**
 - Normalizar variables
 - Coeficientes *comparables*
 - entre variables



Regresión Lineal con **Múltiples** Variables de **Salida**

Regresión Lineal - Versiones

- **Regresión Lineal Simple**
 - **1** variable de entrada
 - **1** variable de salida
 - Modelo
 - $f(x) = m x + b$
- **Regresión Lineal**
 - **m** variables de entrada
 - **1** variable de salida
 - Modelo
 - $f(x) = x[1] w[1] + x[2] w[2] + \dots + x[m] w[m] + b$
- **Regresión Lineal Múltiple**
 - **m** variables de entrada
 - **k** variables de salida

Regresión Lineal Múltiple - Motivación

- Predecir M valores
 - a partir de N valores
- Transformar un vector en otro
 - **Vector de flotantes**
 - codificación **universal**
- Puede usarse para
 - **Predecir** la posición de un objeto
 - a partir de una imagen
 - **Generar** un texto
 - A partir de un sonido
 - **Generar** una imagen
 - A partir de un vector de características
 - Etc...

Estudio	Edad	Promedio	N1	N2
2	24	4	7	7.2
5	22	3	4.5	5.2
7	25	4	6.3	6
9	20	7	5.4	4.4
10	19	4	8.2	9.5
11	20	3	7.2	8.1
13,4	21	5	5,5	10
14	20	3	3	4.3

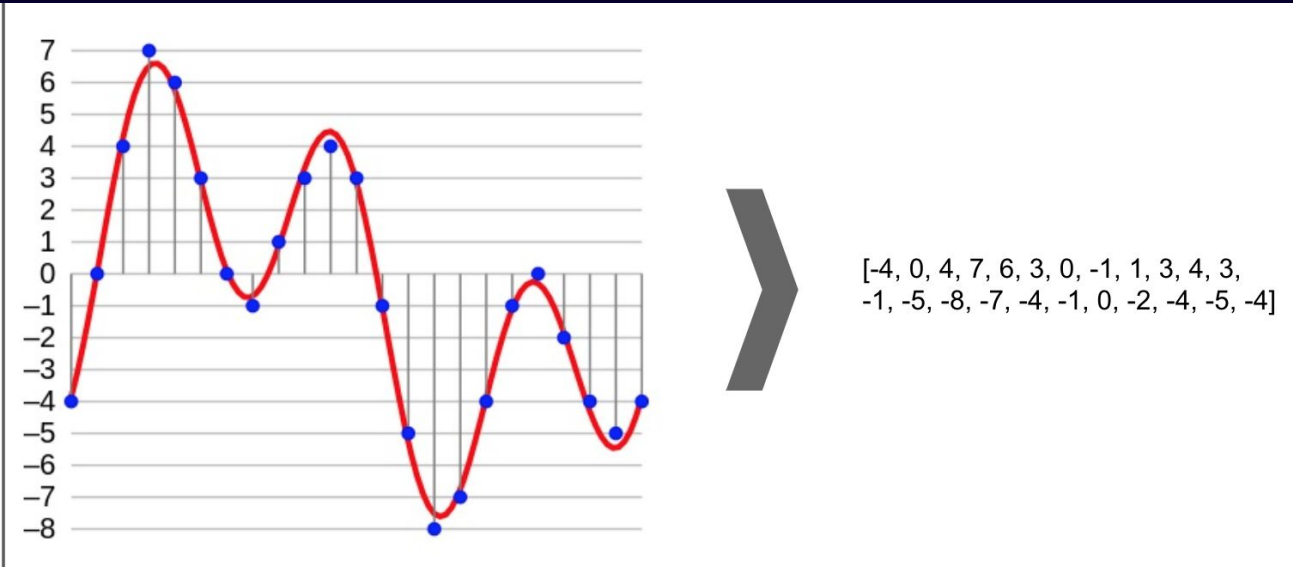
Regresión Lineal Múltiple - Aplicaciones

- **Predecir** la **posición** de un cartel
 - vector de 4 valores: (x,y,alto,ancho)
- **A partir** de una imagen
 - vector de dimensiones: ancho*alto*colores



x	y	alto	ancho
300	200	50	400

Regresión Lineal Múltiple - Aplicaciones



p_1	p_2	p_3	p_4
4	3	2	6

Diccionario



Palabra	Código
hola	1
curso	2
buen	3
que	4
la	5
che	6
el	7

que buen curso che



- **Generar un texto**
 - vector de P dimensiones
- **A partir de un sonido**
 - vector de t dimensiones

Regresión Lineal Múltiple - Aplicaciones

- **Generar** una **imagen**
- **A partir** de un vector de **características**

Perro	Gato	Exterior
1	0	0



Perro	Gato	Exterior
1	0	1

Perro	Gato	Exterior
0	1	0



Perro	Gato	Exterior
1	1	0

Regresión Lineal Múltiple - Problema

Predecir una variable

Estudio	Edad	Promedio	Nota
2	24	4	7
5	22	3	4.5
7	25	4	6.3
9	20	7	5.4
10	19	4	8.2
11	20	3	7.2
13,4	21	5	5,5
14	20	3	3



Predecir múltiples variables

Estudio	Edad	Promedio	N1	N2
2	24	4	7	7.2
5	22	3	4.5	5.2
7	25	4	6.3	6
9	20	7	5.4	4.4
10	19	4	8.2	9.5
11	20	3	7.2	8.1
13,4	21	5	5,5	10
14	20	3	3	4.3

Regresión Lineal Múltiple - Versión Simple

Dos salidas → Dos modelos

Estudio	Edad	Promedio	N1
2	24	4	7
5	22	3	4.5
7	25	4	6.3
9	20	7	5.4
10	19	4	8.2
11	20	3	7.2
13,4	21	5	5,5
14	20	3	3



Modelo para N1

Estudio	Edad	Promedio	N2
2	24	4	7.2
5	22	3	5.2
7	25	4	6
9	20	7	4.4
10	19	4	9.5
11	20	3	8.1
13,4	21	5	10
14	20	3	4.3



Modelo para N2

Regresión Lineal Múltiple - Versión Simple

Modelo para N1

Entrada

X: 3 números

Parámetros

W1: 3 números

B1: 1 número

Salida

N1: 1 número

Fórmula del modelo para N1

$$\begin{aligned} N1(X) &= (X[1] * W1[1] + X[2] * W1[2] + X[3] * W1[3]) + B1 \\ &= X \cdot W1 + B1 \end{aligned}$$

En Python

```
N1 = np.dot(W1,X) + B1
```

$X \cdot W$ es el producto
punto o escalar.

```
def dot(X,W):  
    p=0  
    for i in range(3):  
        p=p+(X[i]*W[i])  
    return p
```

Modelo para N2

Entrada

X: 3 números

Parámetros

W2: 3 números

B2: 1 número

Salida

N2: 1 número

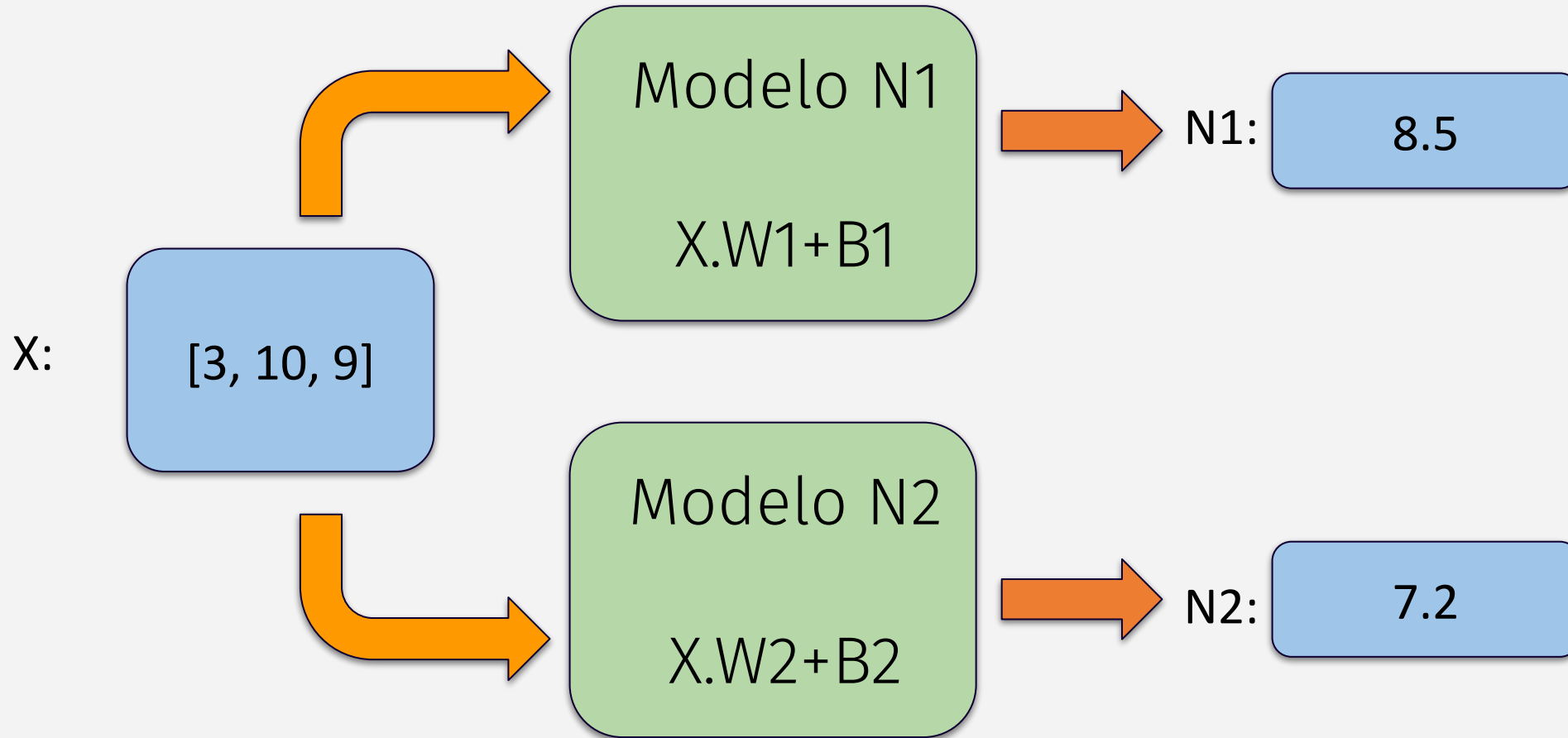
Fórmula del modelo para N2

$$\begin{aligned} N2(X) &= (X[1] * W2[1] + X[2] * W2[2] + X[3] * W2[3]) + B2 \\ &= X \cdot W2 + B2 \end{aligned}$$

En Python

```
N2 = np.dot(W2,X) + B2
```

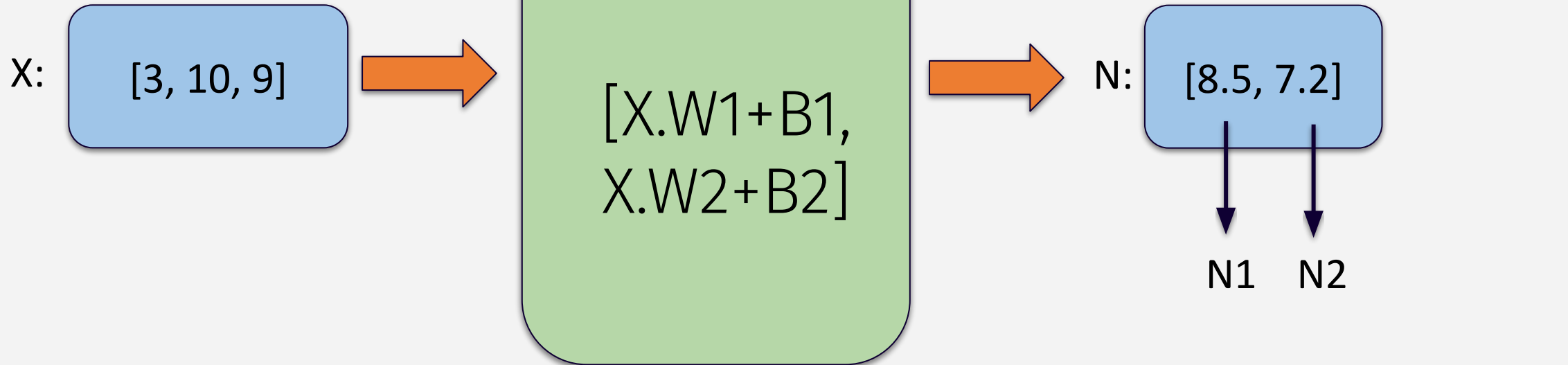
Regresión Lineal Múltiple - Versión Simple



Si tenemos 100 clases?
Hay una forma más eficiente

Regresión Lineal Múltiple - Modelo

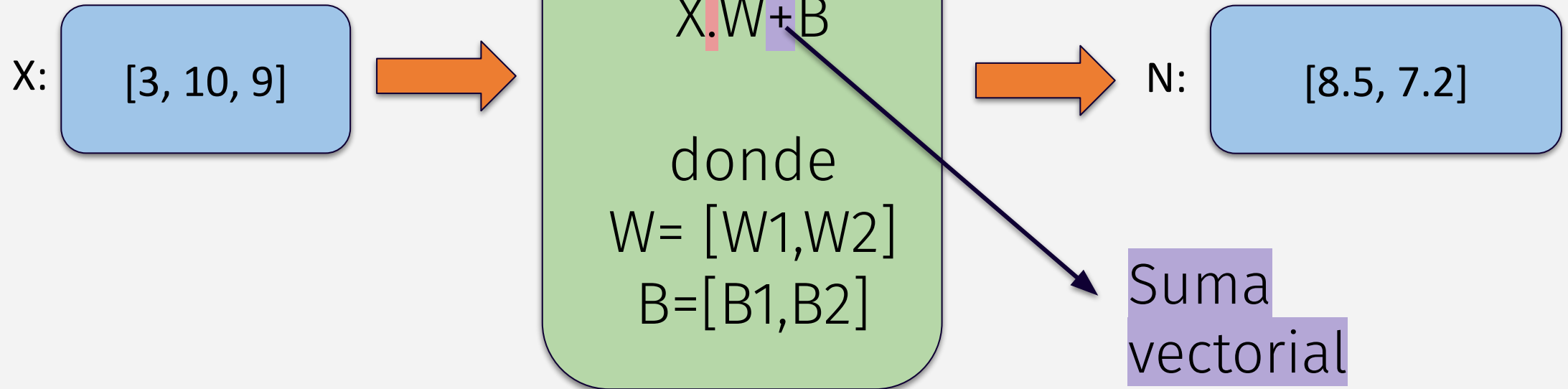
Combinamos N1 y N2
en un solo modelo



Regresión Lineal Múltiple - Modelo

Combinamos los parámetros en:

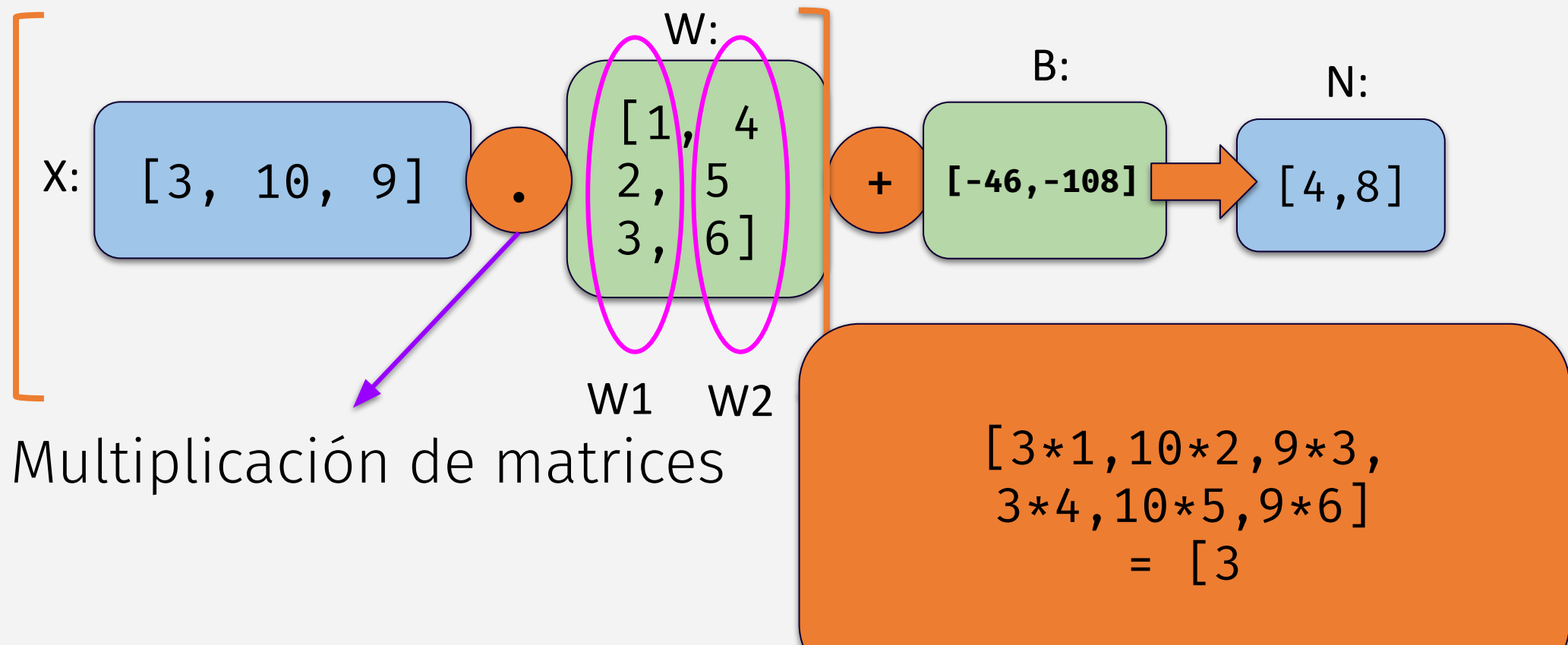
- Matriz W de 3×2
- Vector B de 2×1



Regresión Lineal Múltiple - Cálculo

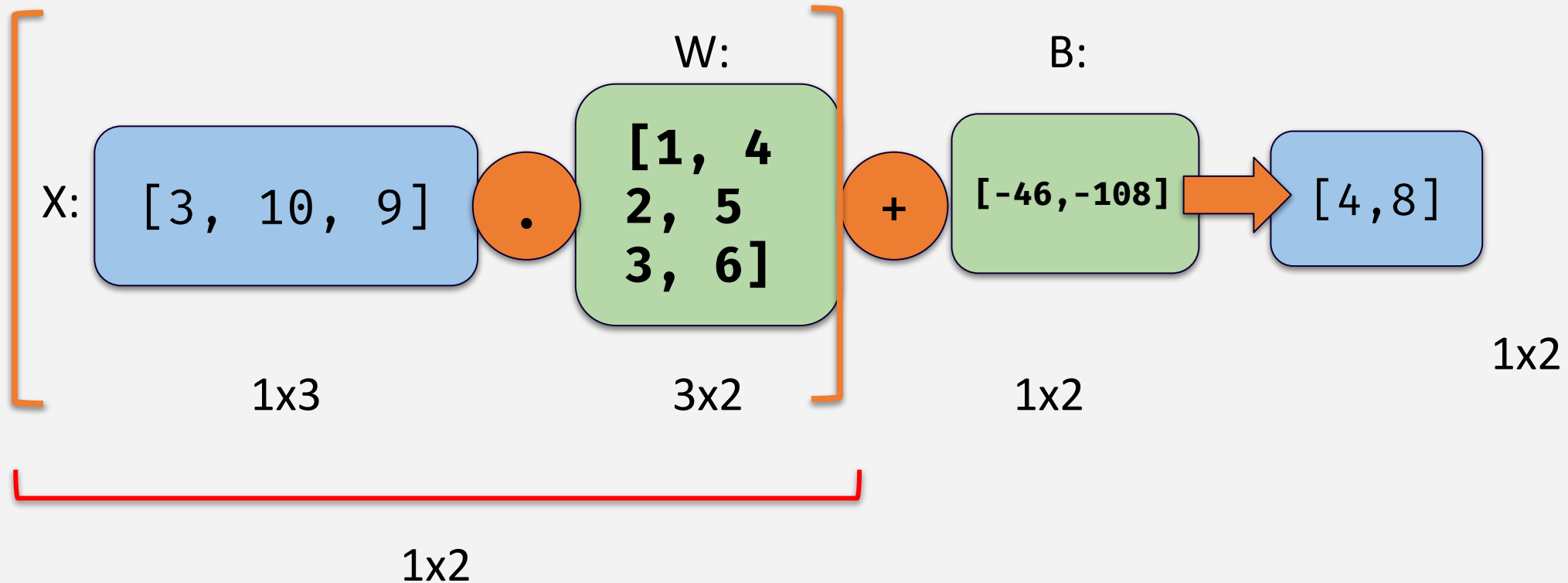
- W1 $[1, 2, 3]'$
- B1 -46

- W2 $[4, 5, 6]'$
- B2 -108



Regresión Lineal Múltiple - Tamaños

Tamaños de vectores y matrices

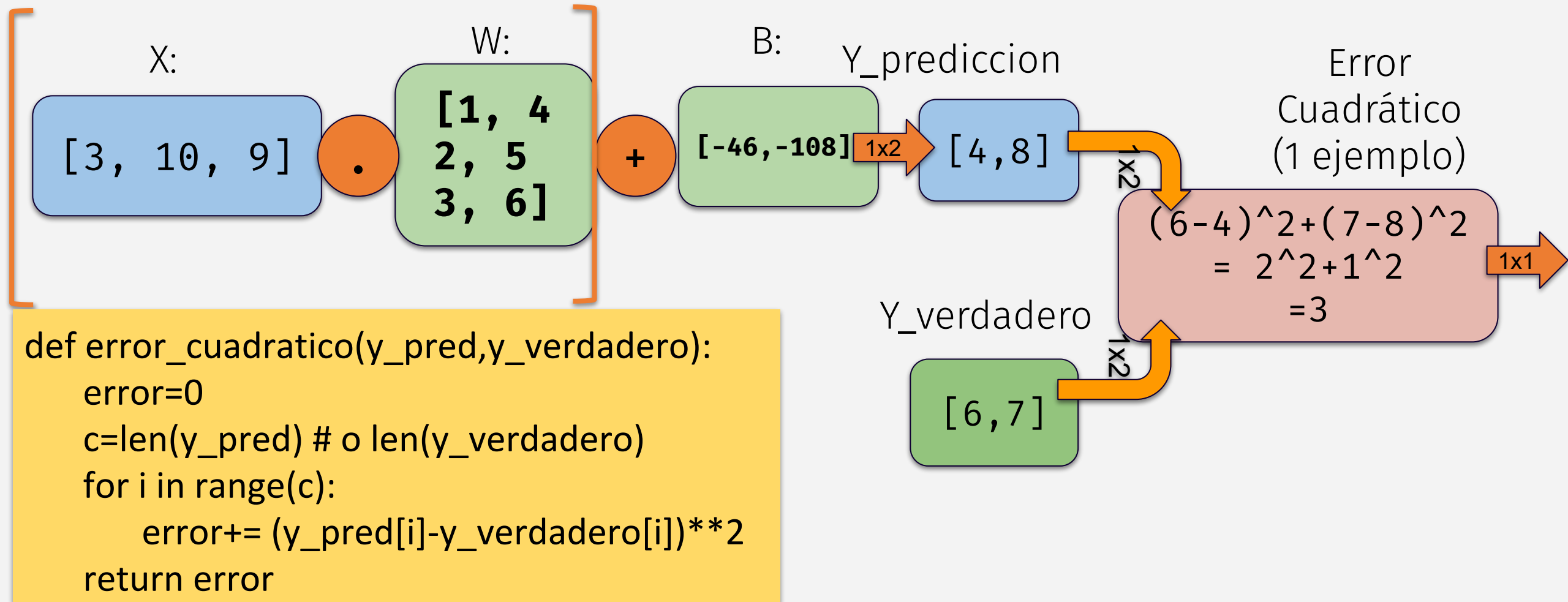


Recursos para repasar operaciones vectoriales

- Producto punto o escalar entre dos vectores:
<https://www.youtube.com/watch?v=N5f7pYTNcFM>
- Multiplicación de un vector por un escalar:
<https://es.khanacademy.org/math/algebra-home/alg-vectors/alg-scalar-multiplication/v/understanding-multiplying-vectors-by-scalars>
- Multiplicación de una matriz por un escalar:
<https://www.youtube.com/watch?v=-ArUqjhQIBM>
- Producto de una matriz y un vector
<https://www.youtube.com/watch?v=2Gdy1xRnqjk>
- Producto de una matriz y otra matriz
<https://www.youtube.com/watch?v=Tjrm3HsqBXE>
<https://es.khanacademy.org/math/linear-algebra/matrix-transformations/composition-of-transformations/v/linear-algebra-matrix-product-examples>

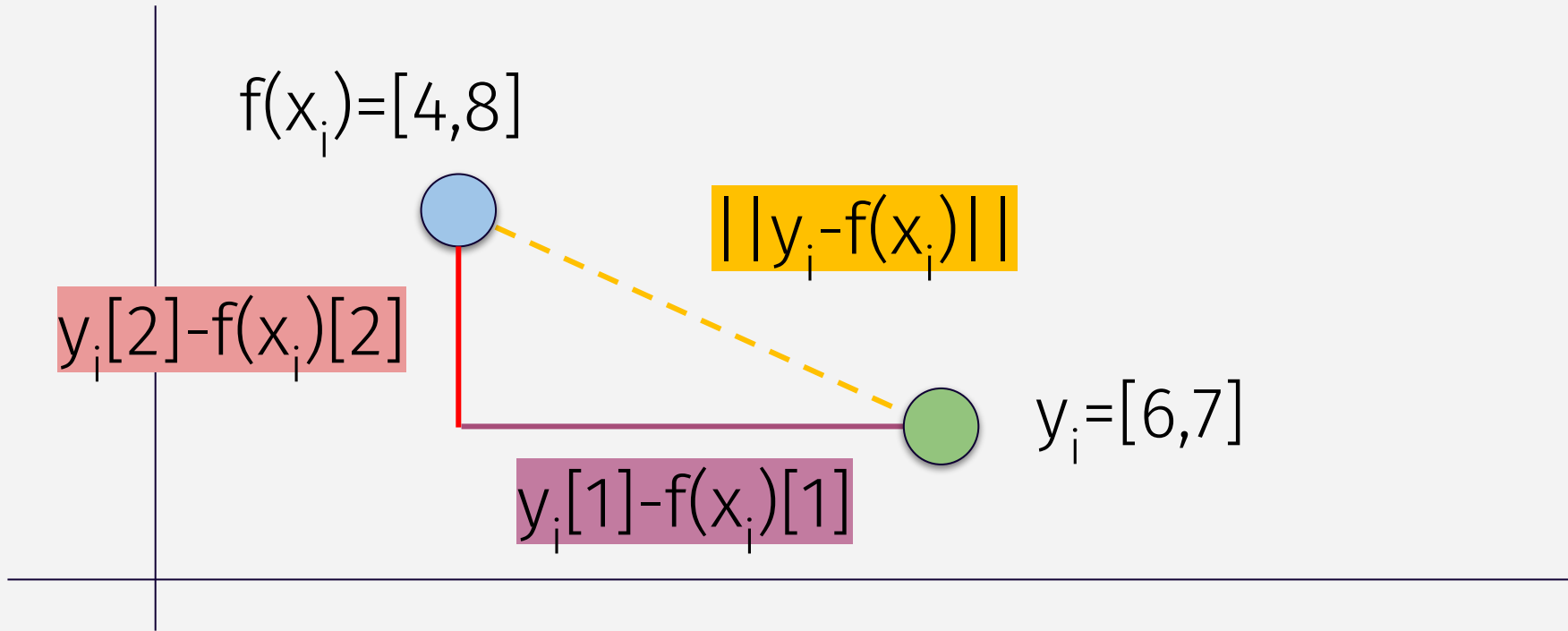
Regresión Lineal Múltiple - Función de Error E_i

$$E_i(\mathbf{W}, \mathbf{B}) = (y_i[1] - f(x_i)[1])^2 + (y_i[2] - f(x_i)[2])^2 = ||y_i - f(x_i)||^2$$



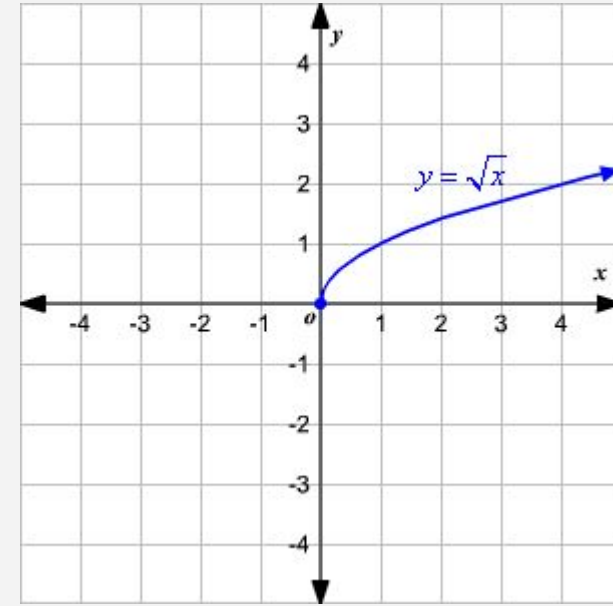
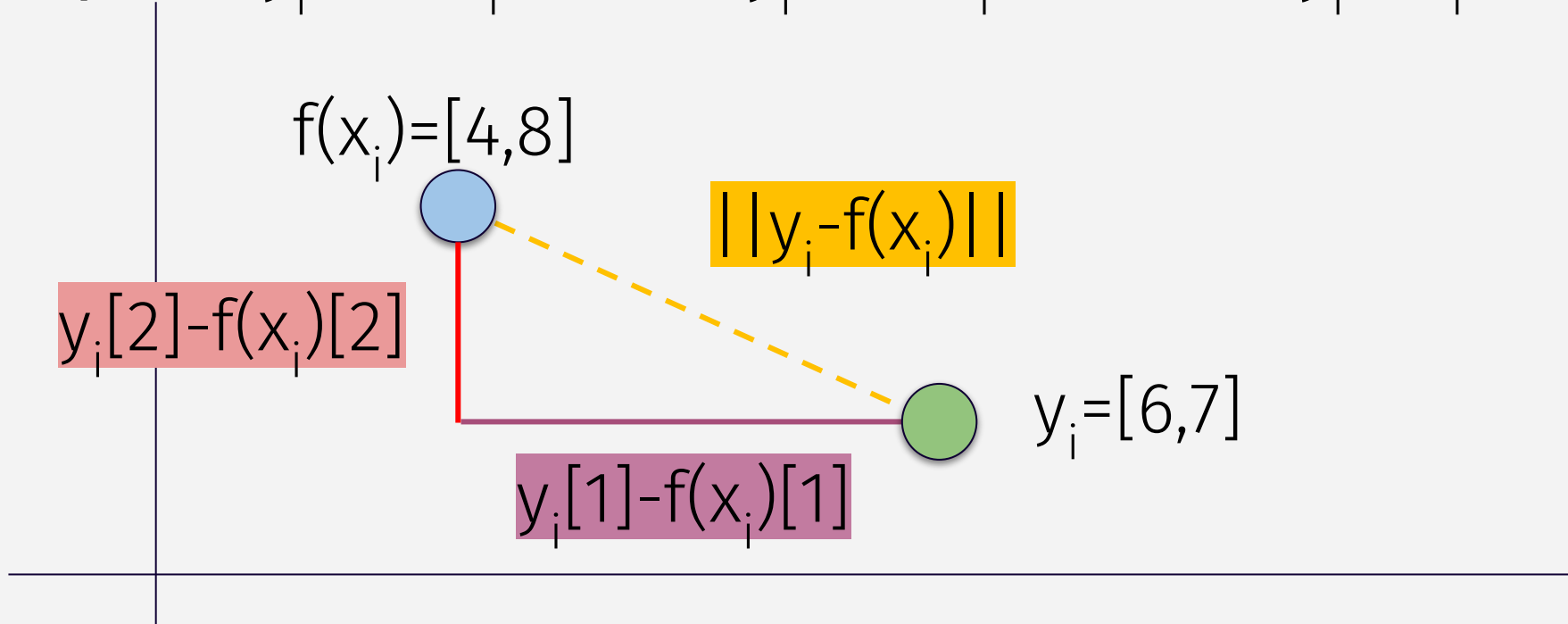
Regresión Lineal Múltiple - Función de Error E_i

- $E_i(\mathbf{W}, \mathbf{B}) = (y_i[1] - f(x_i)[1])^2 + (y_i[2] - f(x_i)[2])^2 = ||y_i - f(x_i)||^2$
 - Distancia euclídea al **cuadrado**
 - Entre $f(x_i)$ e y_i



Regresión Lineal Múltiple - Función de Error E_i

$$E_i(\mathbf{W}, \mathbf{B}) = (y_i[1] - f(x_i)[1])^2 + (y_i[2] - f(x_i)[2])^2 = ||y_i - f(x_i)||^2$$



- E_i = Distancia Euclídea al cuadrado = $||y_i - f(x_i)||^2$
- ¿Por qué no $||y_i - f(x_i)|| = \text{sqrt}[(y_i[1] - f(x_i)[1])^2 + (y_i[2] - f(x_i)[2])^2]$?
 - sqrt es función monótona creciente
 - Misma solución

Regresión Lineal Múltiple - Función de Error **E**

X = (Estudio, Edad, Promedio)

Y_verdadero = (N1,N2)

Y_prediccion = (P1,P2)

$$E(\mathbf{W},\mathbf{B}) = 1/n \sum_i^n E_i(\mathbf{W},\mathbf{B})$$

Estudio	Edad	Promedio	N1	N2	P1	P2	E
2	24	4	7	7.2	16	44	1435.24
5	22	3	4.5	5.2	12	40	1267.29
7	25	4	6.3	6	23	69	4247.89
9	20	7	5.4	4.4	24	70	4649.32
10	19	4	8.2	9.5	14	51	1755.89
11	20	3	7.2	8.1	14	54	2153.05
13.4	21	5	5.5	10	24.4	80.6	5341.57
14	20	3	3	4.3	17	66	4002.89

Promedio(E) = 3106.64

```
def error_cuadratico_medio(x,y,w,b):  
    n = x.shape[0] # o y.shape[0]  
    error=0  
    for i in range(n):  
        P = x[i,:]*w+b #prediccion  
        N = y[i,:]      #verdadero  
        error += error_cuadratico(P,N)  
    return error/n
```

```
def error_cuadratico(y_pred,y_verdadero):  
    error=0  
    c=len(y_pred) # o len(y_verdadero)  
    for i in range(c):  
        error+=(y_pred[i]-y_verdadero[i])**2  
    return error
```

Regresión Lineal Múltiple - Resumen

Resumen

- Predecir N valores
- Transformar un vector en otro
 - Vector de entrada $\mathbf{x}$ de dimension $\mathbf{N}$
 - Vector de salida $\mathbf{y}$ de dimension $\mathbf{M}$
 - Matriz de pesos $\mathbf{W}$, de dimension $\mathbf{N \times M}$
 - Vector de bias $\mathbf{B}$, de dimensión $\mathbf{M}$
- Descenso de gradiente
 - Sumar errores cuadráticos de cada salida.
 - Error sigue siendo convexo (solución única).
- ¿Es más potente que Regresión Lineal Simple?
 - No, sigue siendo un modelo lineal.

Estudio	Edad	Promedio	N1	N2
2	24	4	7	7.2
5	22	3	4.5	5.2
7	25	4	6.3	6
9	20	7	5.4	4.4
10	19	4	8.2	9.5
11	20	3	7.2	8.1
13.4	21	5	5.5	10
14	20	3	3	4.3

Descenso de Gradiente Estocástico

Motivación: Coste computacional de Descenso de Grad.

- Elementos
 - N ejemplos
 - P parámetros
 - I iteraciones
- En cada iteración
 - **Cálculo de $\delta E / \delta w_i$** , $i=1, \dots, P$
 - Asumo *mejor caso*
 - Lineal en N y P: **$O(NP)$**
 - **Actualización de w_i** , $i=1, \dots, M$
 - Lineal en la P: **$O(P)$**
- **Coste total**
 - $O(INP) + O(IP)$
 - **$O(INP)$**

Iteración 1

Horas	Nota
2	1
5	3.2
7	4.5
9	6
10	4
11	4.5
13.4	5.5
14	3
15	5



$$\begin{aligned}w_1 &= w_1 - \delta E / \delta w_1 \\w_2 &= w_2 - \delta E / \delta w_2 \\&\dots \\w_p &= w_p - \delta E / \delta w_p\end{aligned}$$

Iteración 2

Horas	Nota
2	1
5	3.2
7	4.5
9	6
10	4
11	4.5
13.4	5.5
14	3
15	5



$$\begin{aligned}w_1 &= w_1 - \delta E / \delta w_1 \\w_2 &= w_2 - \delta E / \delta w_2 \\&\dots \\w_p &= w_p - \delta E / \delta w_p\end{aligned}$$

Motivación: Coste computacional de Descenso de Grad.

- Elementos
 - N ejemplos
 - P parámetros
 - I iteraciones
- En cada iteración
 - **Cálculo de $\delta E / \delta w_i$** , $i=1, \dots, P$
 - Asumo *mejor caso*
 - Lineal en N y P: **$O(NP)$**
 - **Actualización de w_i** , $i=1, \dots, M$
 - Lineal en la P: **$O(P)$**
- **Coste total**
 - $O(INP) + O(IP)$
 - **$O(INP)$**

```
def descenso_gradiente(E,w,x,y)
    W = ...
    P = len(w)
    converge=False
    while not converge:
        for i in range(P):
             $\delta E \delta W[i]$  = derivada(E,i,x,y)
        for i in range(P):
            w[i] = w[i] -  $\delta E \delta W[i]$ 
        converge = ... (depende)
    return w
```

Entrenamiento por lotes

- Coste total **$O(INP)$**
 - Cálculo de derivadas **$O(NP)$**

- **Idea**

- Dividir datos en **lotes**
 - de **$B \ll N$** ejemplos
- Calcular derivadas con **lotes**

- Gradiente aproximado

- Ruidoso

- Pero funciona

- Cálculo de derivadas

- **$O(BP)$**

- Ejemplo con **$B=3$**

- Épocas = recorridos del conjunto de datos
- Iteraciones = act. de los **w_i**

Época 1

Horas	Nota
2	1
5	3.2
7	4.5
9	6
10	4
11	4.5
13.4	5.5
14	3
15	5

Iteración 1
(lote 1)

2	1
5	3.2
7	4.5

Iteración 2
(lote 2)

9	6
10	4
11	4.5

Iteración 3
(lote 3)

13.4	5.5
14	3
15	5

$$\begin{aligned}w_1 &= w_1 - \delta E / \delta w_1 \\w_2 &= w_2 - \delta E / \delta w_2 \\&\dots \\w_p &= w_p - \delta E / \delta w_p\end{aligned}$$

$$\begin{aligned}w_1 &= w_1 - \delta E / \delta w_1 \\w_2 &= w_2 - \delta E / \delta w_2 \\&\dots \\w_p &= w_p - \delta E / \delta w_p\end{aligned}$$

$$\begin{aligned}w_1 &= w_1 - \delta E / \delta w_1 \\w_2 &= w_2 - \delta E / \delta w_2 \\&\dots \\w_p &= w_p - \delta E / \delta w_p\end{aligned}$$

Época 2

Horas	Nota
2	1
5	3.2
7	4.5
9	6
10	4
11	4.5
13.4	5.5
14	3
15	5

Iteración 1
(lote 1)

2	1
5	3.2
7	4.5

Iteración 2
(lote 2)

9	6
10	4
11	4.5

Iteración 3
(lote 3)

13.4	5.5
14	3
15	5

$$\begin{aligned}w_1 &= w_1 - \delta E / \delta w_1 \\w_2 &= w_2 - \delta E / \delta w_2 \\&\dots \\w_p &= w_p - \delta E / \delta w_p\end{aligned}$$

$$\begin{aligned}w_1 &= w_1 - \delta E / \delta w_1 \\w_2 &= w_2 - \delta E / \delta w_2 \\&\dots \\w_p &= w_p - \delta E / \delta w_p\end{aligned}$$

$$\begin{aligned}w_1 &= w_1 - \delta E / \delta w_1 \\w_2 &= w_2 - \delta E / \delta w_2 \\&\dots \\w_p &= w_p - \delta E / \delta w_p\end{aligned}$$

Descenso de gradiente Clásico vs Por lotes

- Asumimos
 - model.w = matriz con todos los parámetros de la red
 - δw = matriz con la derivada de respecto a model.w
- Descenso de gradiente Clásico

```
x,y = load_data()

lr = 0.001 # learning rate
model = ...
epochs = 1000

for i in range(epochs):
     $\delta w = \text{model.derivatives}(x,y)$ 
     $\text{model.w} = \text{model.w} - \text{lr} * \delta w$ 
```

- Descenso de gradiente por lotes

```
x,y = load_data()

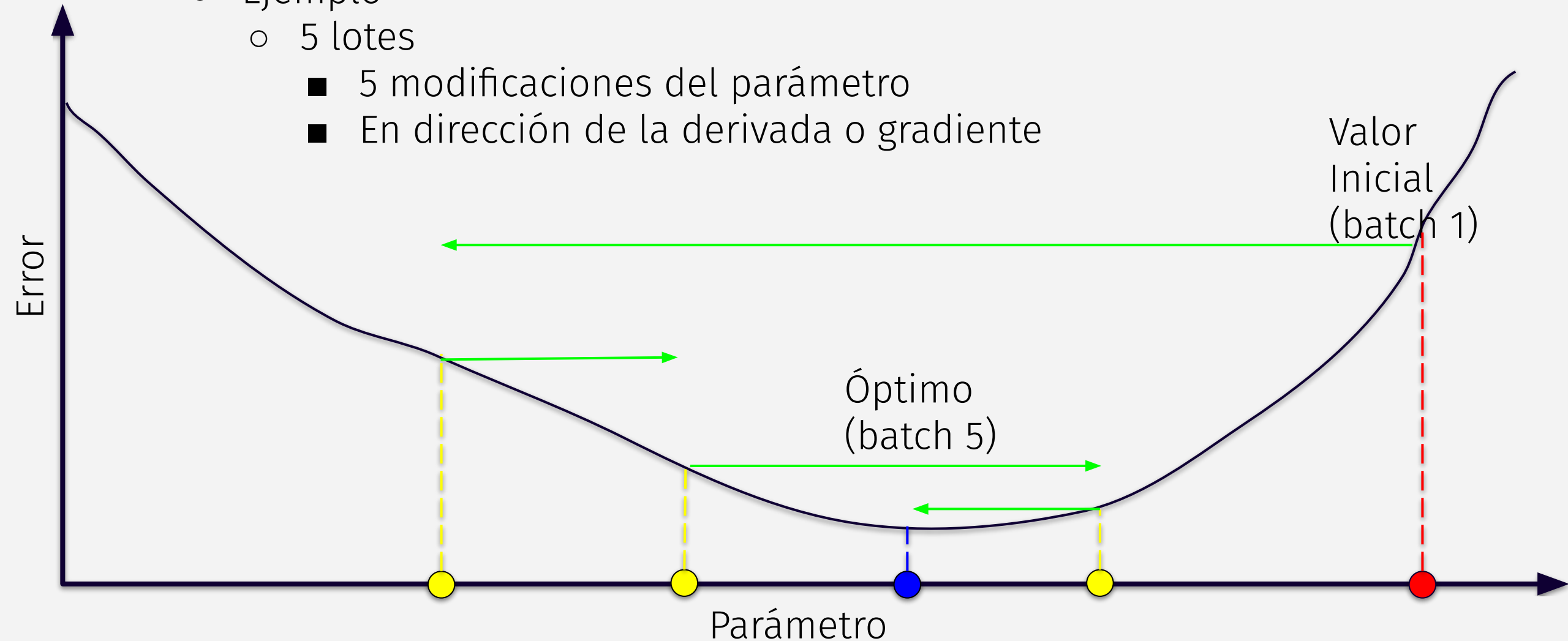
lr = 0.001 # learning rate
model = ...
epochs = 1000

n = x.shape[0]
batch_size = 32
batches = n // batch_size

for i in range(epochs):
    for batch in range(batches):
        batch_x, batch_y = get_batch(x,y,batch)
         $\delta w = \text{model.derivatives}(\text{batch\_x}, \text{batch\_y})$ 
         $\text{model.w} = \text{model.w} - \text{lr} * \delta w$ 
```

Descenso de gradiente por lotes

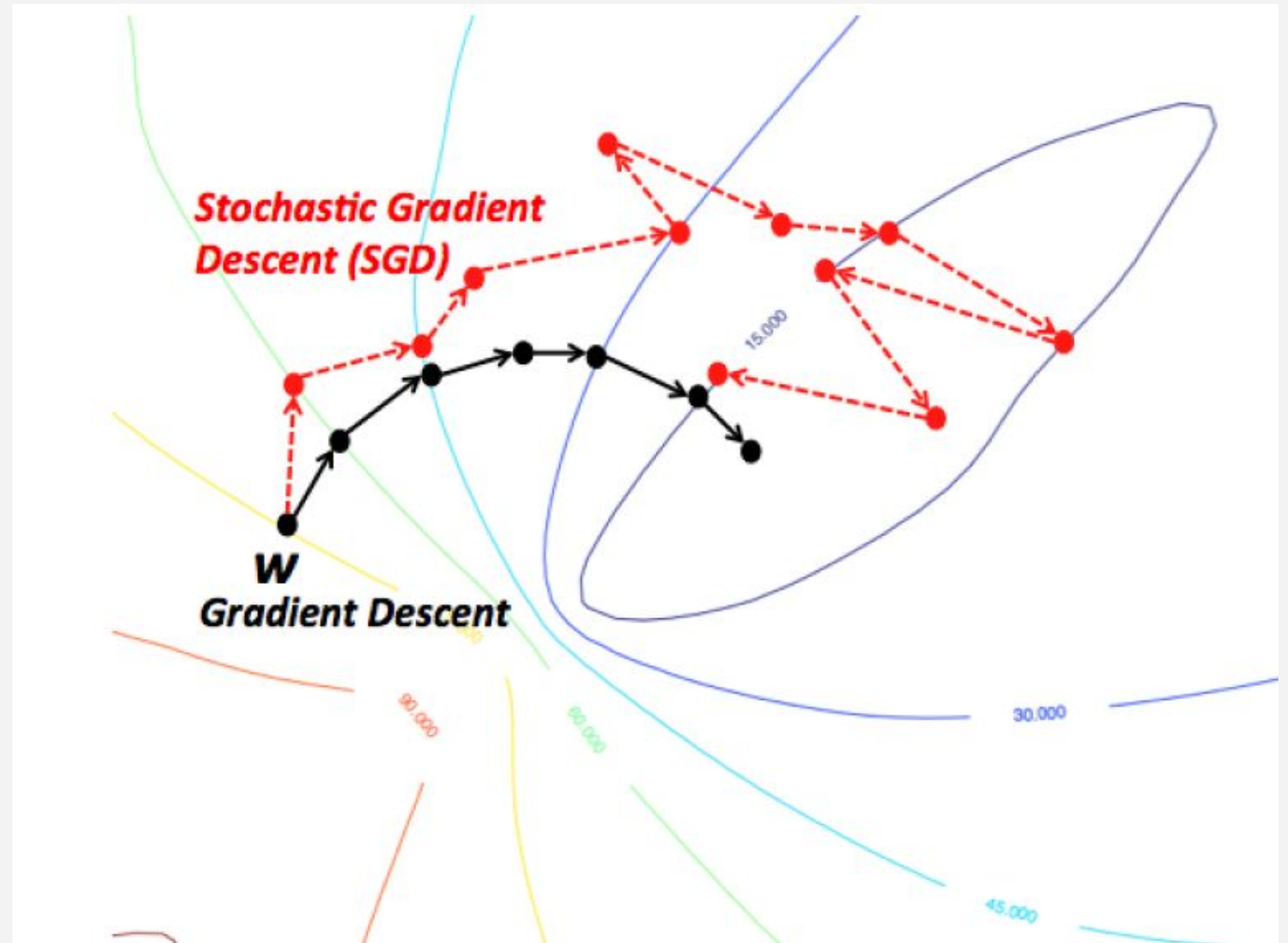
- Ejemplo
 - 5 lotes
 - 5 modificaciones del parámetro
 - En dirección de la derivada o gradiente



Coste computacional: Descenso de gradiente

estocástico

- Descenso clásico
 - Coste de iteración
 - **$O(NP)$**
- Descenso por lotes
 - Coste de iteración
 - **$O(BP)$**
 - **B** es un hiperparámetro
- Si **$B = N$**
 - Igual que clásico
- Si **$B = 1$**
 - Gradientes demasiado ruidoso
- Si $B =$ intermedio
 - Gradientes ruidosos
 - Pero buenos
 - Más pasos/iteraciones
 - Pero más rápidas

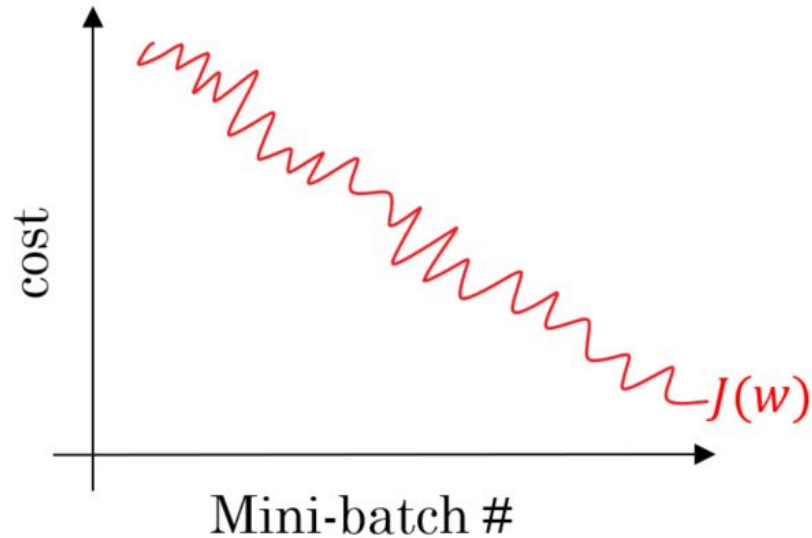


Por lotes = ruidoso = **estocástico**

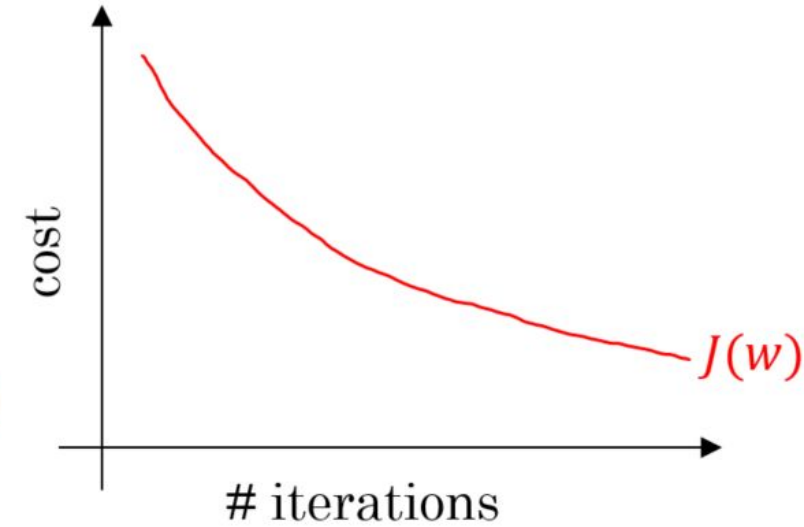
Descenso de Gradiente Estocástico - Efecto

- Error (cost)
 - Cambia de forma “ruidosa”
 - Algunas iteraciones aumentan el error
 - Pero decrece en promedio
- (minibatch = estocástico)
- (batch = clásico)

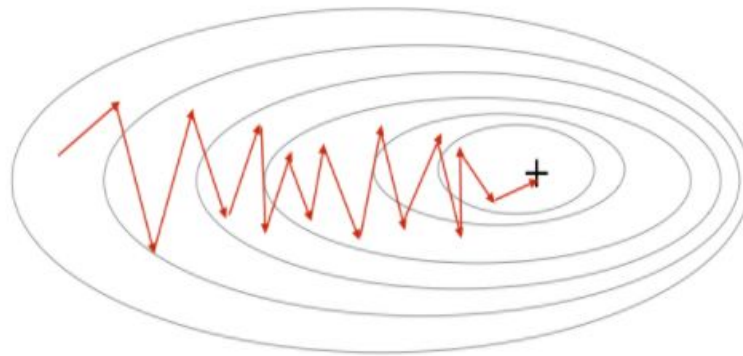
Mini-batch gradient descent



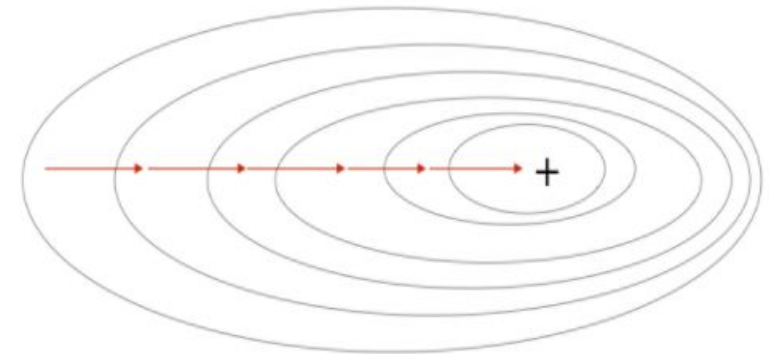
Batch gradient descent



Stochastic Gradient Descent

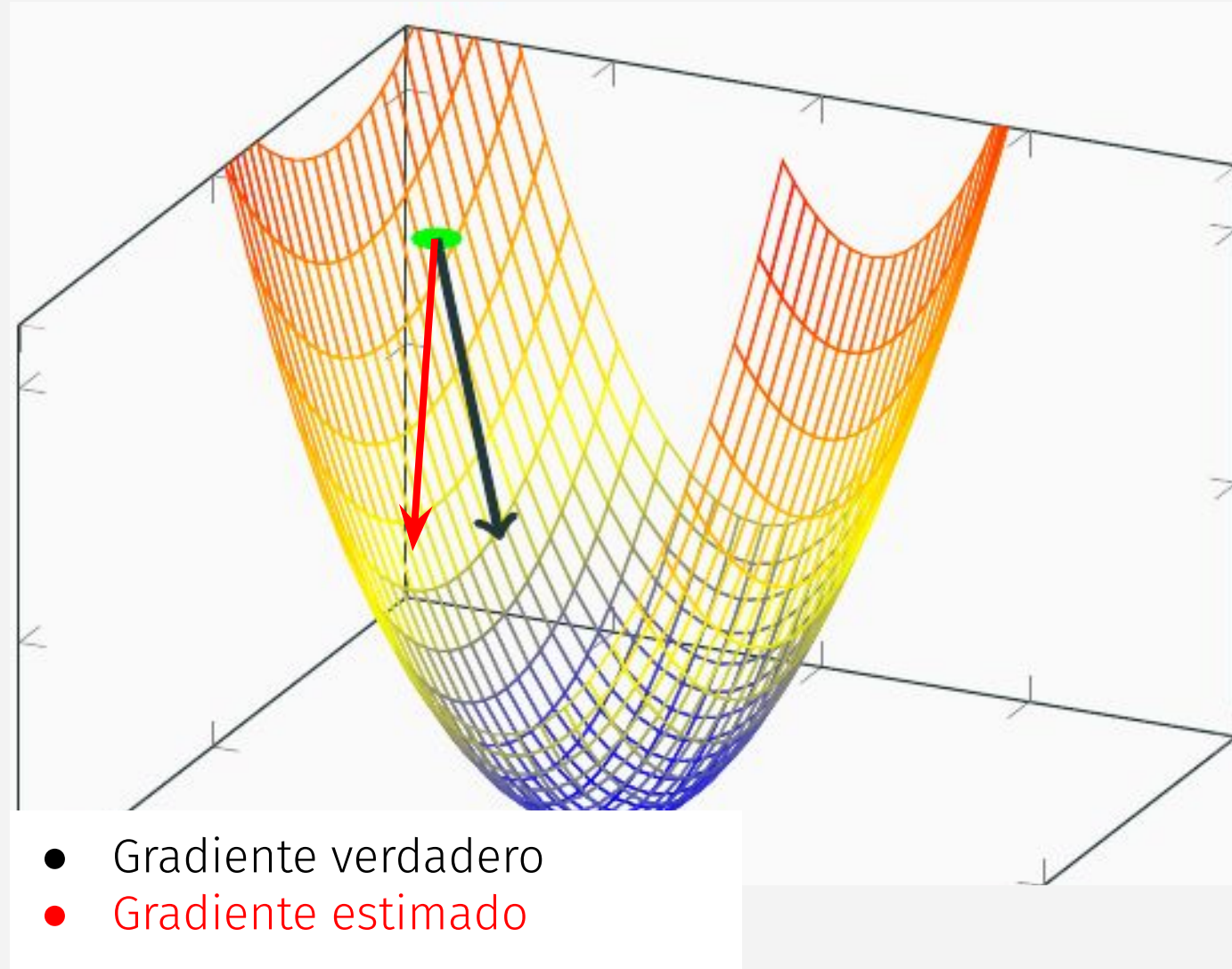


Gradient Descent



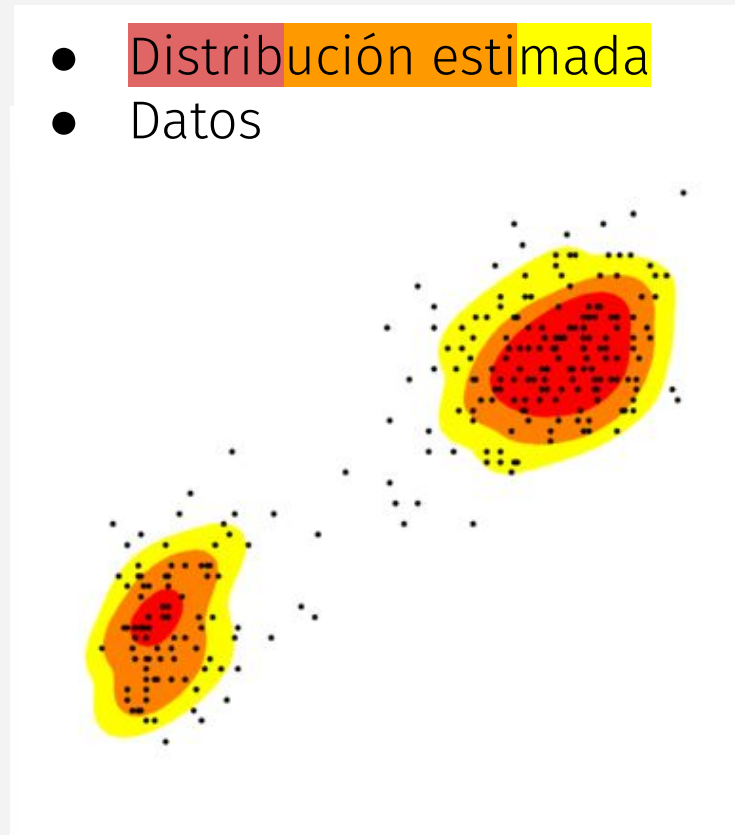
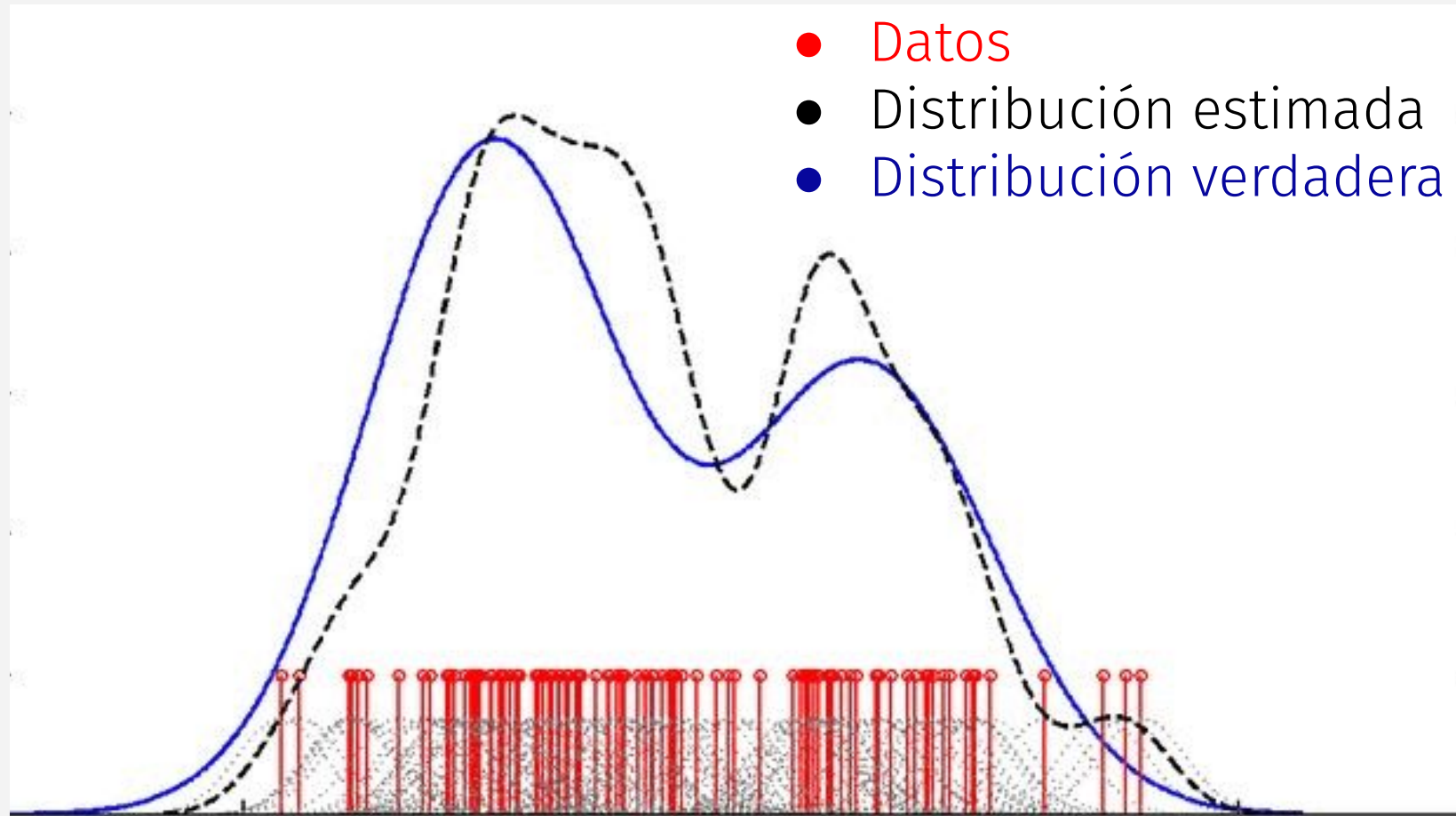
¿Por qué funciona?

- **Descenso de Gradiente Clásico**
 - Utiliza todas las **muestras**
 - Gradiente es exacto
 - Respecto de las **muestras**
 - Siguen siendo **muestras**
 - **No es exacto (es ruidoso)**
 - Respecto de la distribución
- **Descenso de Gradiente Estocástico**
 - Utiliza una **muestra** de la **muestra**
 - Gradiente **no** es **exacto**
 - **Significativo estadísticamente**
 - **Útil para entrenar**



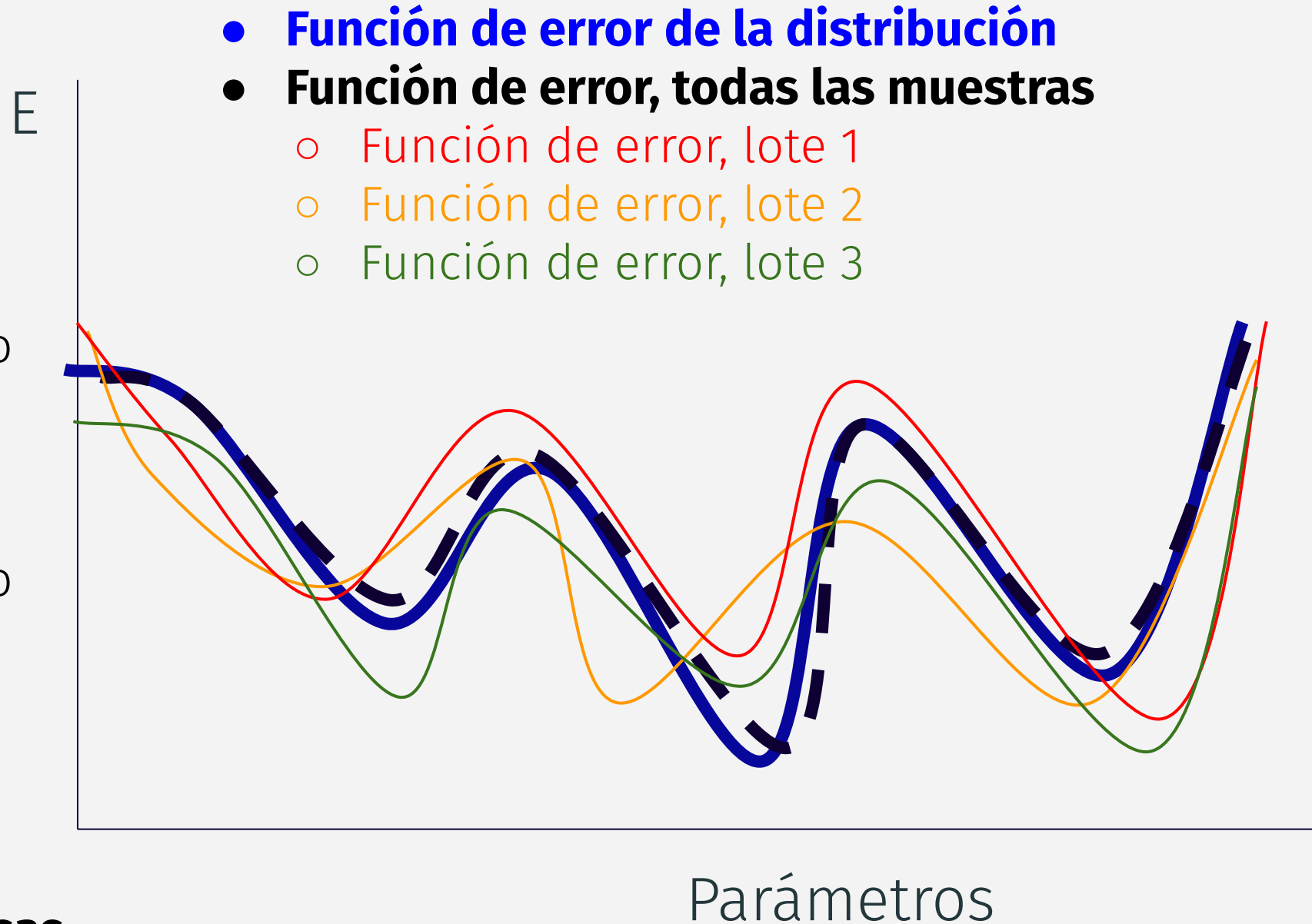
¿Por qué funciona?

- Distribuciones vs muestras
 - La **distribución** nunca se completamente
 - La estimamos a través de **muestras**



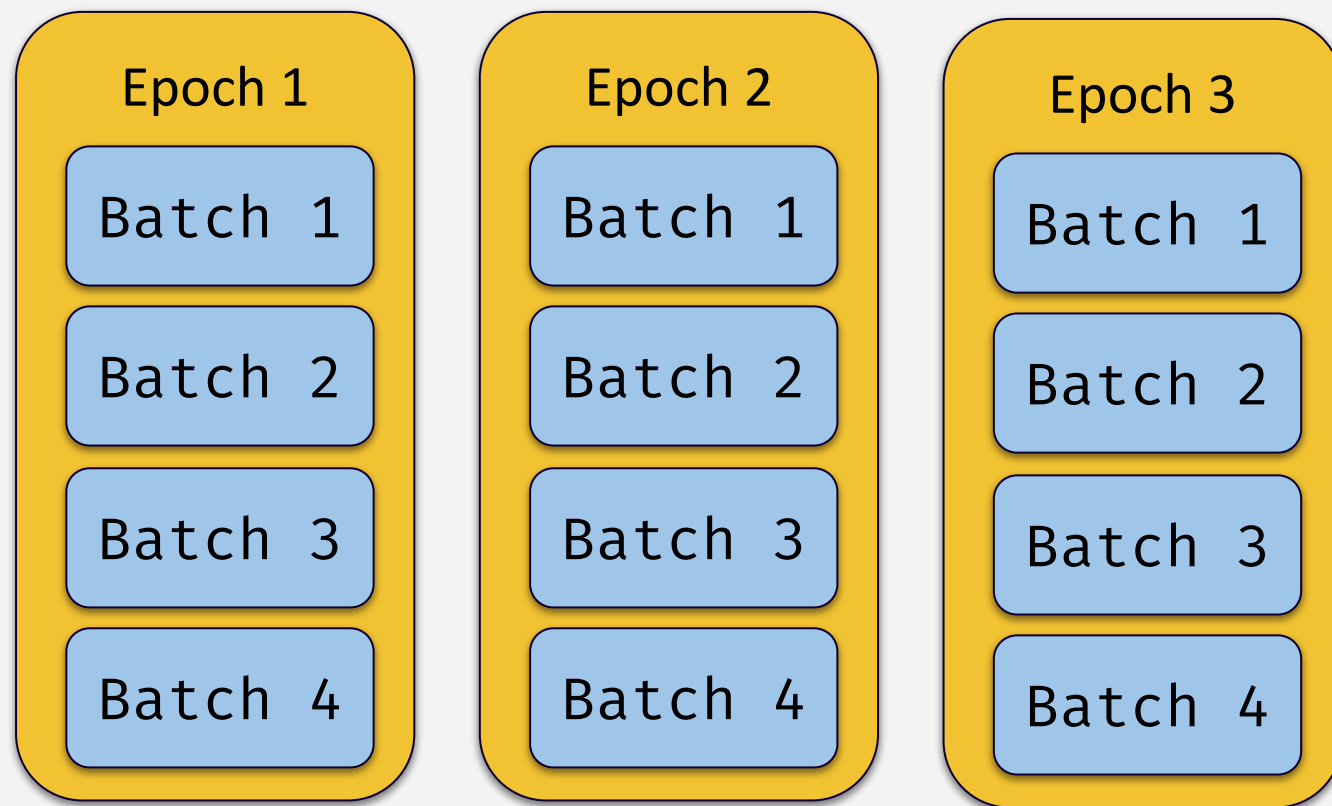
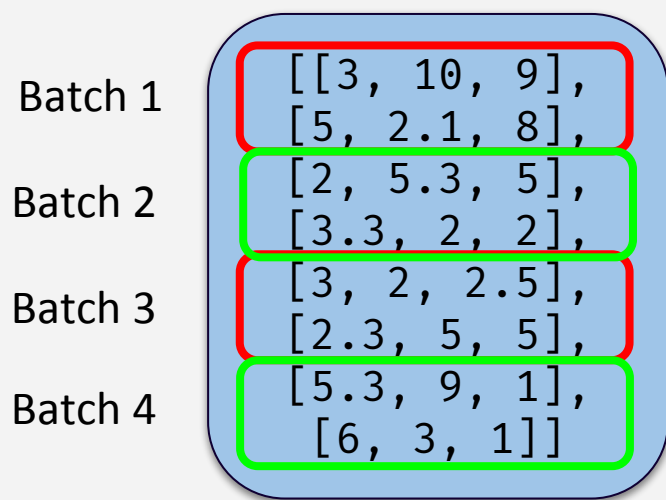
¿Por qué funciona?

- Función de error de la distribución
 - Imposible de conocer
- Función de error de la muestra completa
 - Utilizada por Descenso Clásico
- Función de error de cada lote
 - Utilizada por Descenso Estocástico
 - **Cada lote redefine la función de error**
 - Se **redefinen** las **derivadas**
 - **Equivalente: ruidosas**



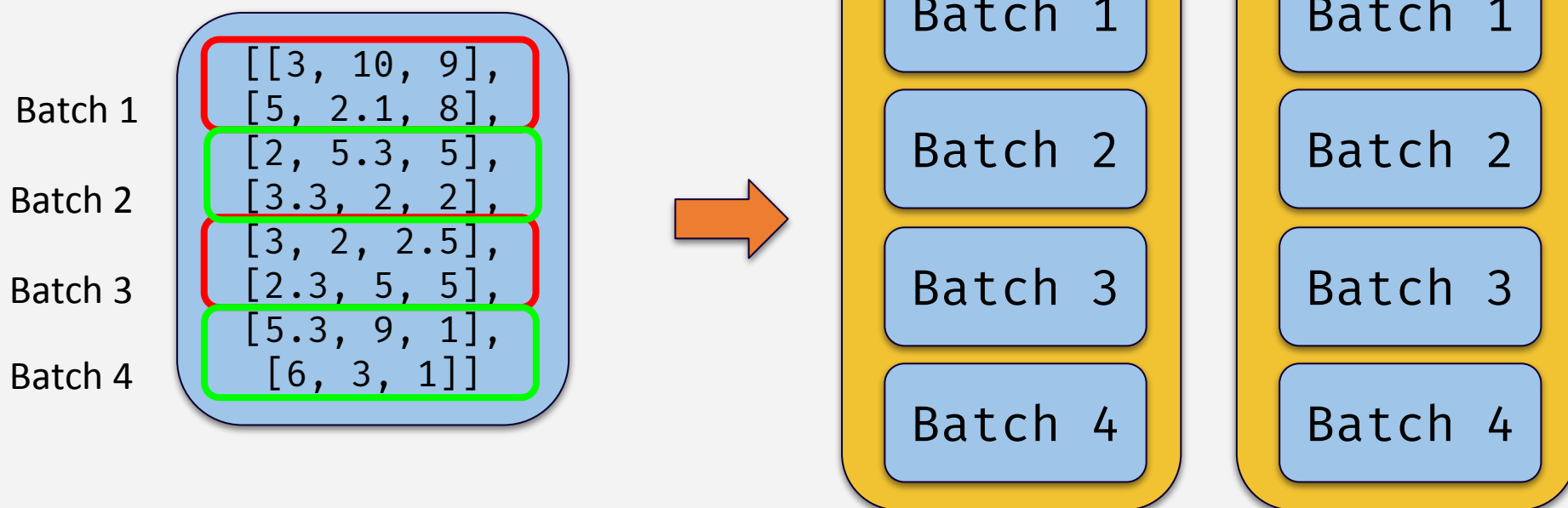
Épocas, tamaños de lote e iteraciones

- Épocas = epochs
 - Cantidad de iteraciones de entrenamiento
 - En relación al tamaño del dataset
- Ejemplo con **N=8, batch_size=2, y epochs = 3**
 - Iteraciones reales por epoch = 4
 - $N/\text{batch_size}$
 - Iteraciones reales totales = 12
 - $N/\text{batch_size} * \text{epochs}$



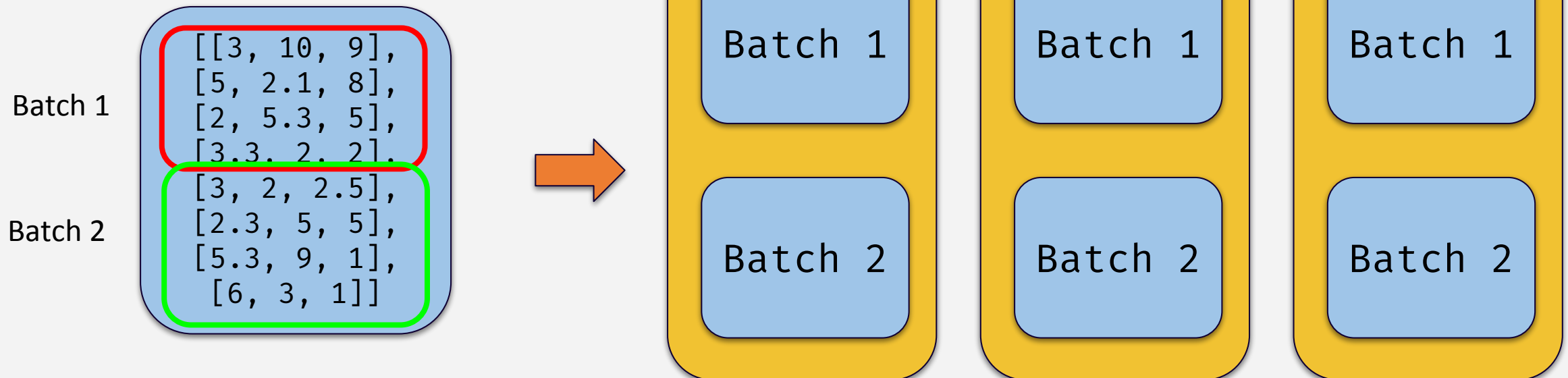
Épocas, tamaños de lote e iteraciones

- Épocas = epochs
 - Cantidad de iteraciones de entrenamiento
 - En relación al tamaño del dataset
- Ejemplo con **N=8, batch_size=2, y epochs = 2**
 - Iteraciones reales por epoch = 4
 - $N/\text{batch_size}$
 - Iteraciones reales totales = 8
 - $N/\text{batch_size} * \text{epochs}$



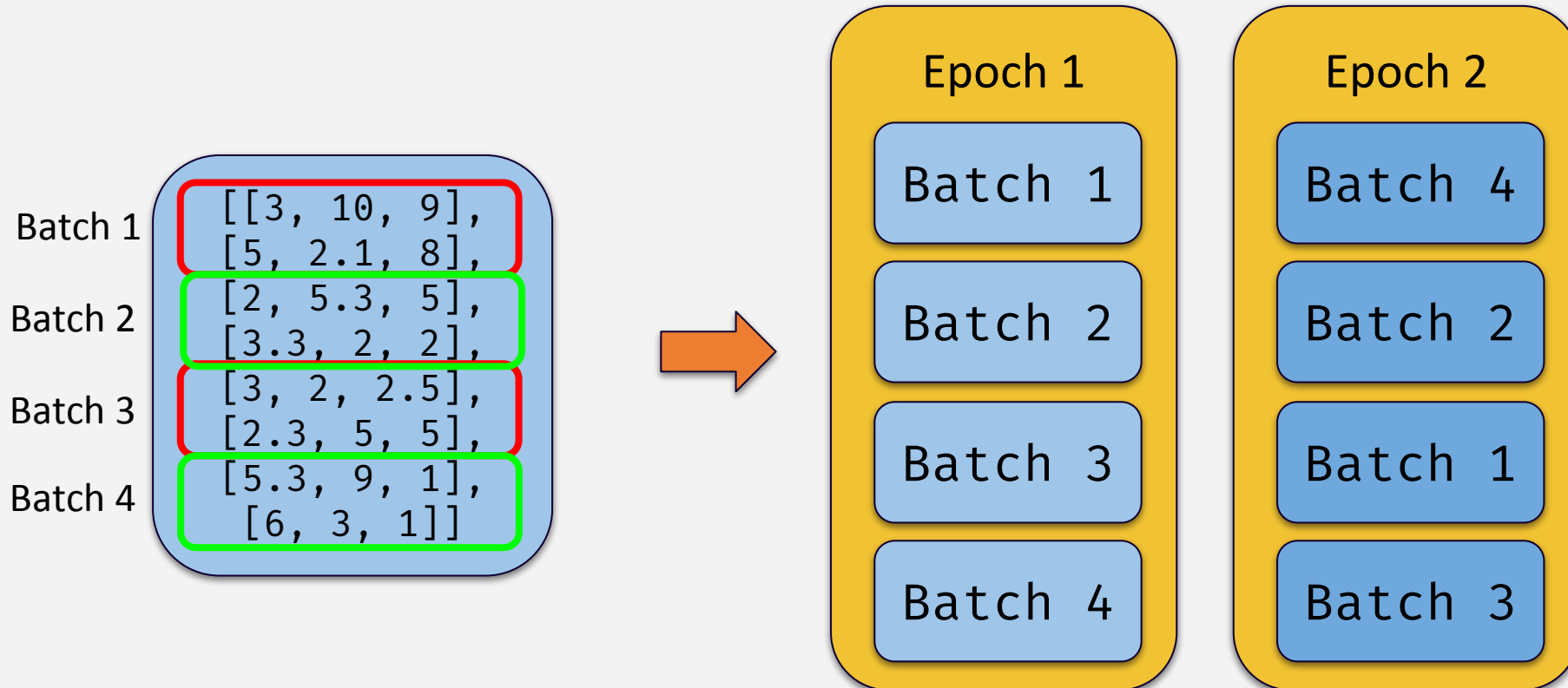
Épocas, tamaños de lote e iteraciones

- Épocas = epochs
 - Cantidad de iteraciones de entrenamiento
 - En relación al tamaño del dataset
- Ejemplo con **N=8, batch_size=4, y epochs = 3**
 - Iteraciones reales por epoch = 2
 - $N/\text{batch_size}$
 - Iteraciones reales totales = 6
 - $N/\text{batch_size} * \text{epochs}$



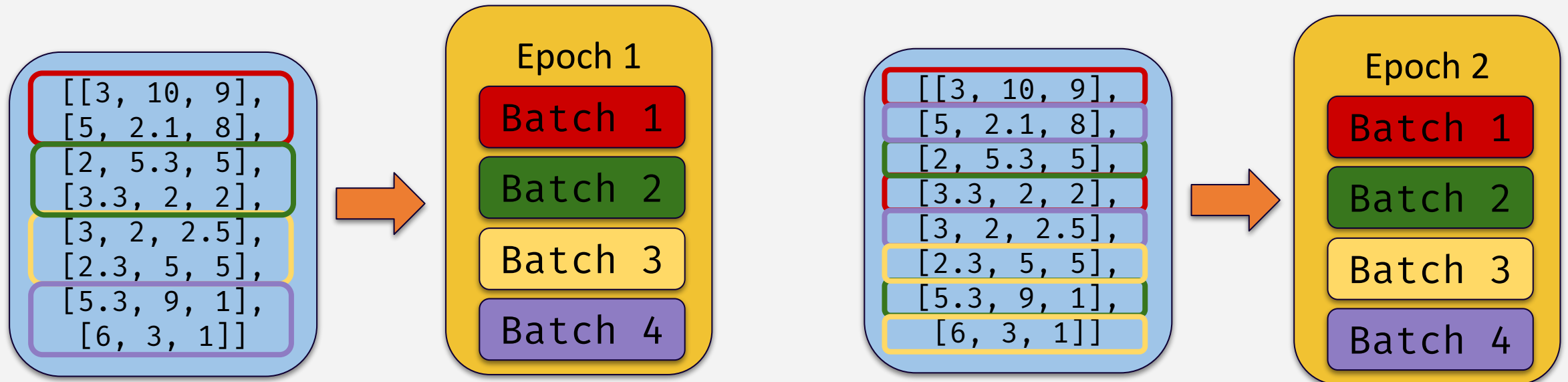
Desordenar lotes entre épocas

- Ayuda al entrenamiento
 - Genera ruido
 - Remueve efectos de orden



Desordenar ejemplos entre épocas

- Mismo objetivo que desordenar lotes
- Mismo efecto
 - Distinto orden de ejemplos
 - Distintos lotes
 - Aún más desorden



Paréntesis: nomenclatura

- Lote = Batch
 - Tamaño de lote = Batch Size = B
- Épocas = Epochs
 - Algunos las llaman **iteraciones** 🧑
- Descenso de Gradiente **Tradicional**
 - *También conocido como*
 - Descenso de Gradiente **Batch**
 - (Batch Gradient Descent)
 - Pero **NO** utiliza lotes 🧑
 - Proceso **Batch** vs **Online** (fábricas)

Paréntesis: nomenclatura

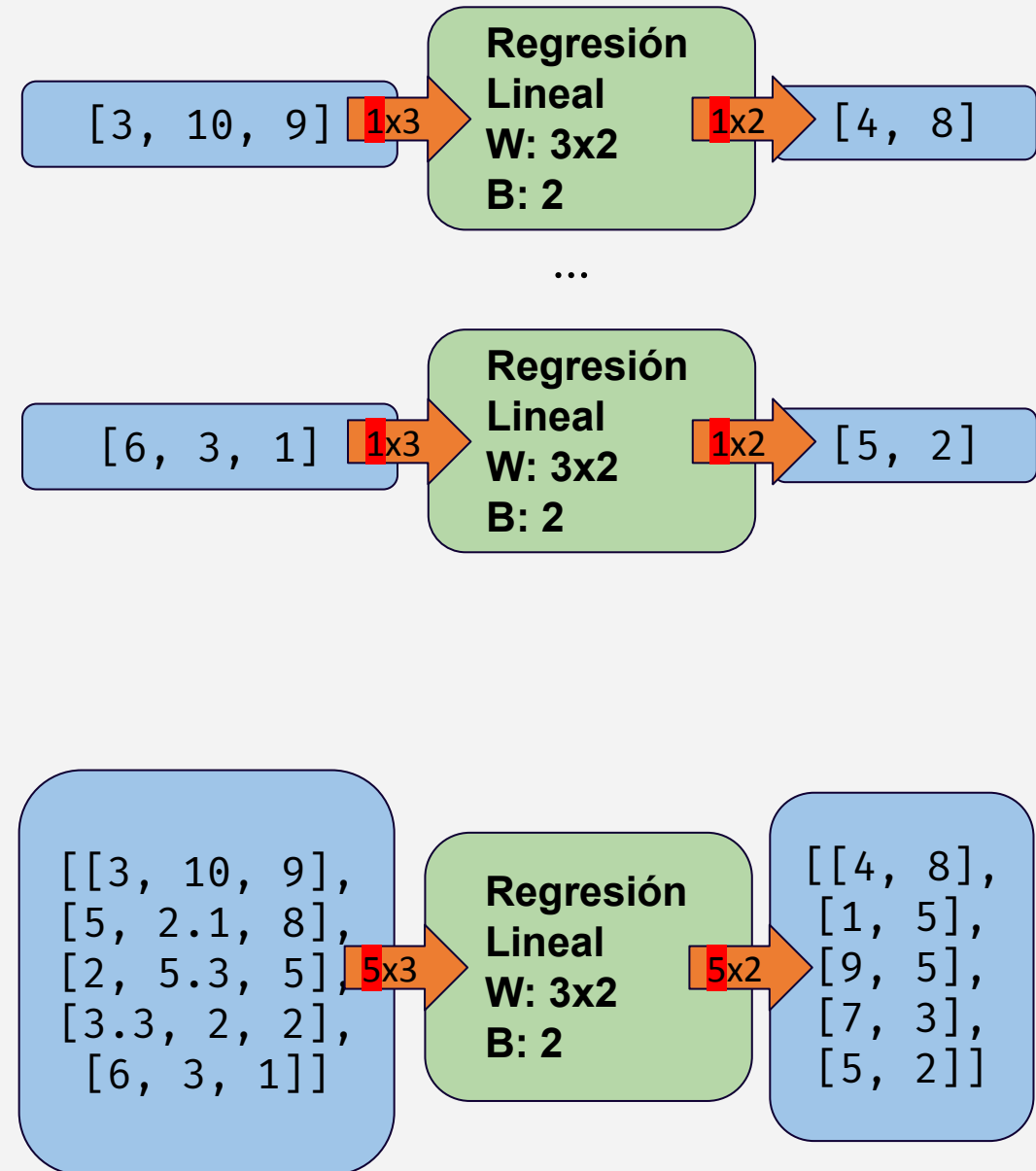
- Descenso de Gradiente **Estocástico**
 - Utiliza **lotes/batches**
 - *También conocido como*
 - Descenso de Gradiente **mini-batch** 🙋
- Descenso de Gradiente **Online**
 - Utiliza lotes de tamaño 1
 - $B = 1$
 - Simula reentrenar el modelo con nuevos ejemplos continuamente
 - *También conocido como*
 - Descenso de Gradiente **Estocástico** 🙋

Tamaño de lote

- **Predicción** por lotes
 - Más eficiente
 - Multiplicaciones de matrices
 - pre-alocación de memoria
 - Con GPUs:
 - Maximiza el uso de la memoria
 - Minimiza overhead copias GPU $\leftrightarrow$ CPU
- Tamaño de lote
 - No se puede usar todo el dataset al mismo tiempo!
 - $N = O(\text{millones})$
 - No entra en memoria
- Generalmente **batch_size** = potencia de 2
 - 8, 16, 32, 64, 128, 256, 512
 - Más de 512 es redundante para entrenar

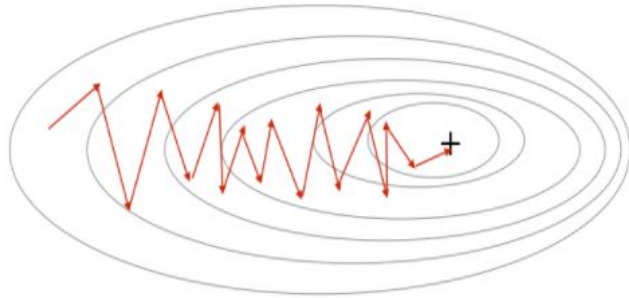
Predicción por lotes

- **Predicción de a un ejemplo**
 - Multiplicación de matrices muy pequeñas
 - Overhead de copias
 - Mala localidad de memoria
 - Con GPUs:
 - overhead copias GPU $\leftrightarrow$ CPU
- **Predicción por lotes**
 - Resuelve estos problemas
 - Tamaño de lote
 - Tan grande como sea posible
 - Tamaño RAM

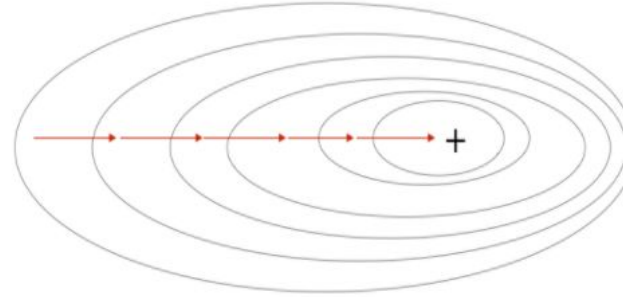


Resumen

Stochastic Gradient Descent



Gradient Descent



Descenso de gradiente tradicional	Descenso de gradiente Estocástico
Gradiente con todos los ejemplos	Gradiente con lotes de ejemplos
Gradientes más exactos	Gradientes ruidosos
No puede escapar de mínimos locales	Puede que escape de mínimos locales
Si f convexa, garantiza mínimo global	Mínimo global no garantizado
Lento, poco escalable	Rápido, escalable

Regresión Lineal con **Keras**

- Librería de Redes Neuronales Profundas
 - Objetivo: Facilidad de uso
 - Prototipado rápido de modelos
 - Modelos pre-entrenados
 - Python
 - Alto nivel
 - Programar a nivel de **capas**
 - **Capa ~= Transformación Entrenable**
 - Regresión Lineal = 1 capa
 - Backend
 - Implementa operaciones eficientes
 - TensorFlow (defecto)
 - Theano o CNTK (ya no se usan)

Definición de un modelo



```
from keras.models import Sequential
from keras.layers import Dense
#Crear un modelo de capas secuenciales
model = Sequential()
```

```
#Agregar capa Densa
model.add(Dense(1, input_shape=[4]))
```

- Dense: $f(x) = wx+b$
- 4 variables de entrada
- 1 variables de salida

```
# compilar el modelo
model.compile(optimizer= 'sgd', loss= 'mse')
```

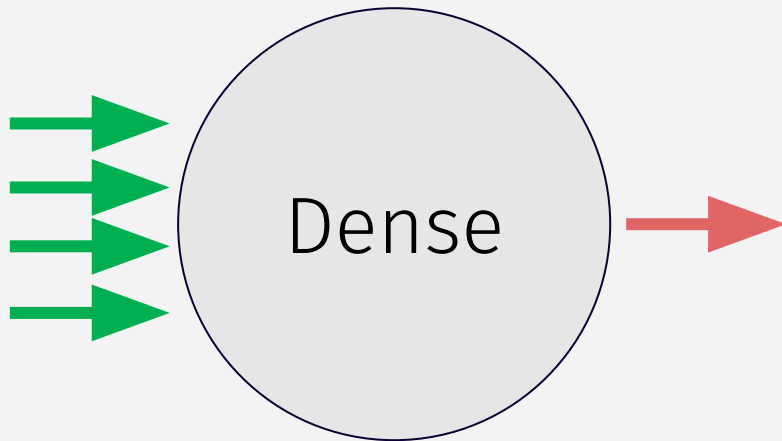
- 'sgd': Descenso de Gradiente Estocástico
- 'mse': Error Cuadrático Medio

```
# Imprimir resumen del modelo
model.summary()
```

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 1)	5
Total params: 5		
Trainable params: 5		
Non-trainable params: 0		

Capa Dense

```
from keras.models import Sequential
from keras.layers import Dense
model = Sequential()
model.add(Dense(1, input_shape=[4]))
```



Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 1)	5
Total params: 5		
Trainable params: 5		
Non-trainable params: 0		

- 5 parámetros
 - 4 de w
 - 4 variables de entrada
 - 1 de salida
 - $w: 4 \times 1$
 - 1 de b
 - 1 de salida
 - $b: 1 \times 1$

```
# cargar datos de alguna forma
# X e Y deben ser matrices de NumPy
X,Y = cargar_dataset()

# entrenar el modelo con 50 épocas
# tamaño de lote 32 y datos X e Y
history = model.fit(X, Y, epochs=50,
batch_size=32)

# Predecir la salida de X
Y_predicted = model.predict(X)

# Calcula el score del modelo
E = model.evaluate(X, Y)
print(f"El error del modelo es {E}")
```

- Métodos de model
 - fit: entrena el modelo
 - predict: calcula la salida del modelo $f(x)$
 - evaluate: calcula el error del modelo (y otras métricas)
- history: error al final de cada época

Calculando las dimensiones a partir de X e Y



```
# Datos
X,Y = cargar_dataset()
nx,d_in = X.shape # X tiene tamaño n x d_in
ny,d_out = Y.shape # Y tiene tamaño n x d_out
assert(nx=ny) # misma cantidad de ejemplos en ambos vectores
```

```
#Modelo
import keras
model=keras.Sequential()
# Capa lineal que convierte vector de d_in a d_out dims
model.add(keras.Dense(d_out,input_shape=[d_in]))
model.compile(loss='mse', optimizer='sgd')
```

```
# Entrenamiento y evaluación
model.fit(X,Y,epochs=100,batch_size=32,shuffle=True)
E=model.evaluate(X,Y)
```

Tensorflow y Keras

Tensorflow vs Keras

Keras

```
# MODELO
model=keras.Sequential(
    keras.Dense(4,activation='
softmax')])

# ENTRENAMIENTO
model.compile(loss='mse',
optimizer='sgd')
model.fit(x,y,epochs=100)

# PREDICCION
y_predicted=model.predict(x)
```

Tensorflow

```
# MODELO

# Variables de entrada
varX = tf.placeholder("float")
varY = tf.placeholder("float")

# Parámetros
W = tf.Variable(np.random.randn())
b = tf.Variable(np.random.randn())

# Salida
y_abs = tf.add(tf.multiply(X, W), b)
y_pred=tf.softmax(y_abs)
```

Tensorflow vs Keras

Keras

```
# MODELO
model=keras.Sequential(
    keras.Dense(4,activation
='softmax' )]

# ENTRENAMIENTO
model.compile(loss='mse',
optimizer='sgd')
model.fit(x,y,epochs=100)

# PREDICCIÓN
y_predicted=model.predict(
x)
```

Tensorflow

```
# ENTRENAMIENTO
errors=tf.nn.sigmoid_cross_entropy_with_logi
ts(logits=y_pred, labels=varY)
error=tf.reduce_mean(errors)

opt = tf.train.GradientDescentOptimizer(
    learning_rate=0.001).minimize(error)

## varias líneas de sarasa ##

# epocas y batches “a mano”
epochs=1000
for i in range(epochs):
    for (batch_x, batch_y) in zip(x, y):
        sess.run(opt,
            feed_dict={varX:batch_x,varY:batch_y})
```

Tensorflow vs Keras

Keras

```
# MODELO
model=keras.Sequential(
    keras.Dense(4,activation
='softmax' )]

# ENTRENAMIENTO
model.compile(loss='mse',
optimizer='sgd')
model.fit(x,y,epochs=100)

# PREDICCIÓN
y_predicted=model.predict(
x)
```

Tensorflow

```
# PREDICCIÓN

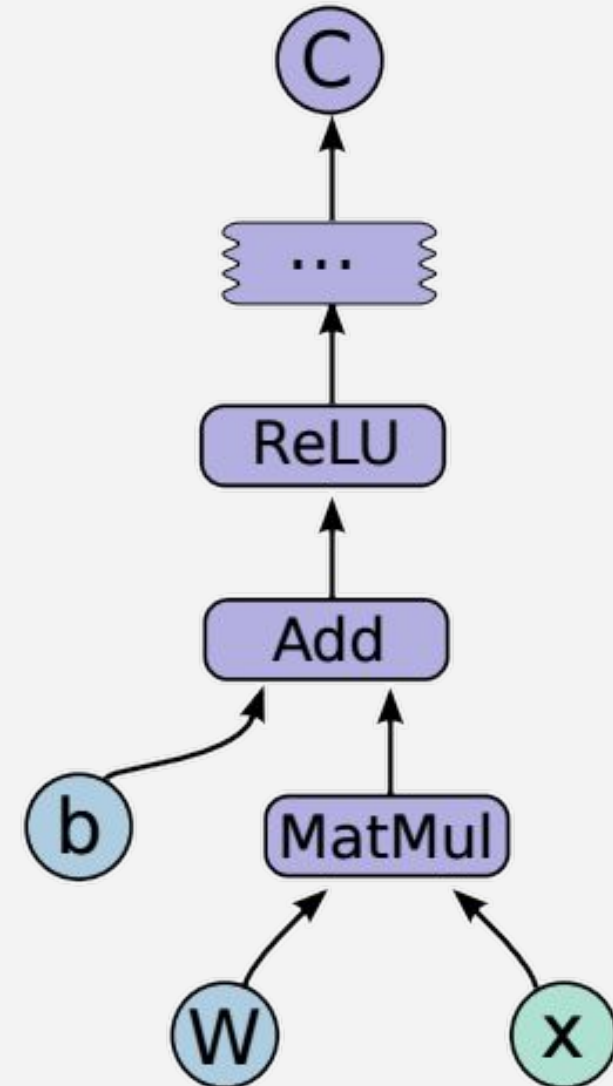
y_pred_value = sess.run(y_pred,
feed_dict={varX:x})
```

Tensorflow - Grafos Declarativos

- Operaciones de tensorflow **declarativas**
 - NO se ejecutan inmediatamente
 - Definen grafos
 - Ejecución mediante `sess.run(op)`
- Desacoplan definición de la ejecución
 - Como una función!
 - Permite optimizar el grafo
- Ejemplo:

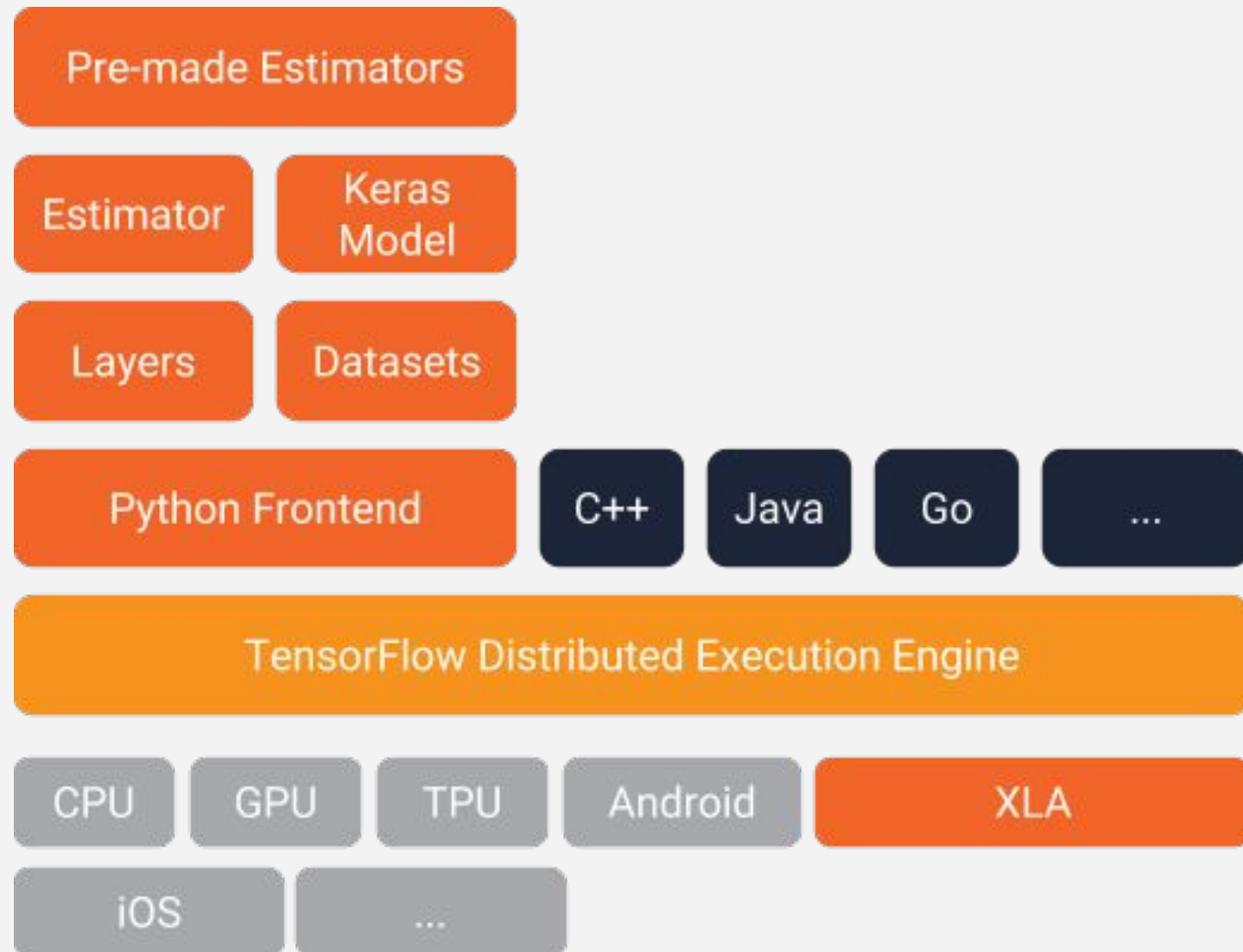
```
x = tf.placeholder("float")
W = tf.Variable(np.random.randn())
b = tf.Variable(np.random.randn())

# Salida
C = tf.nn.relu(tf.add(tf.multiply(x, W), b))
C_value = sess.run(C, feed_dict={x: INPUT })
# vectores numpy
```



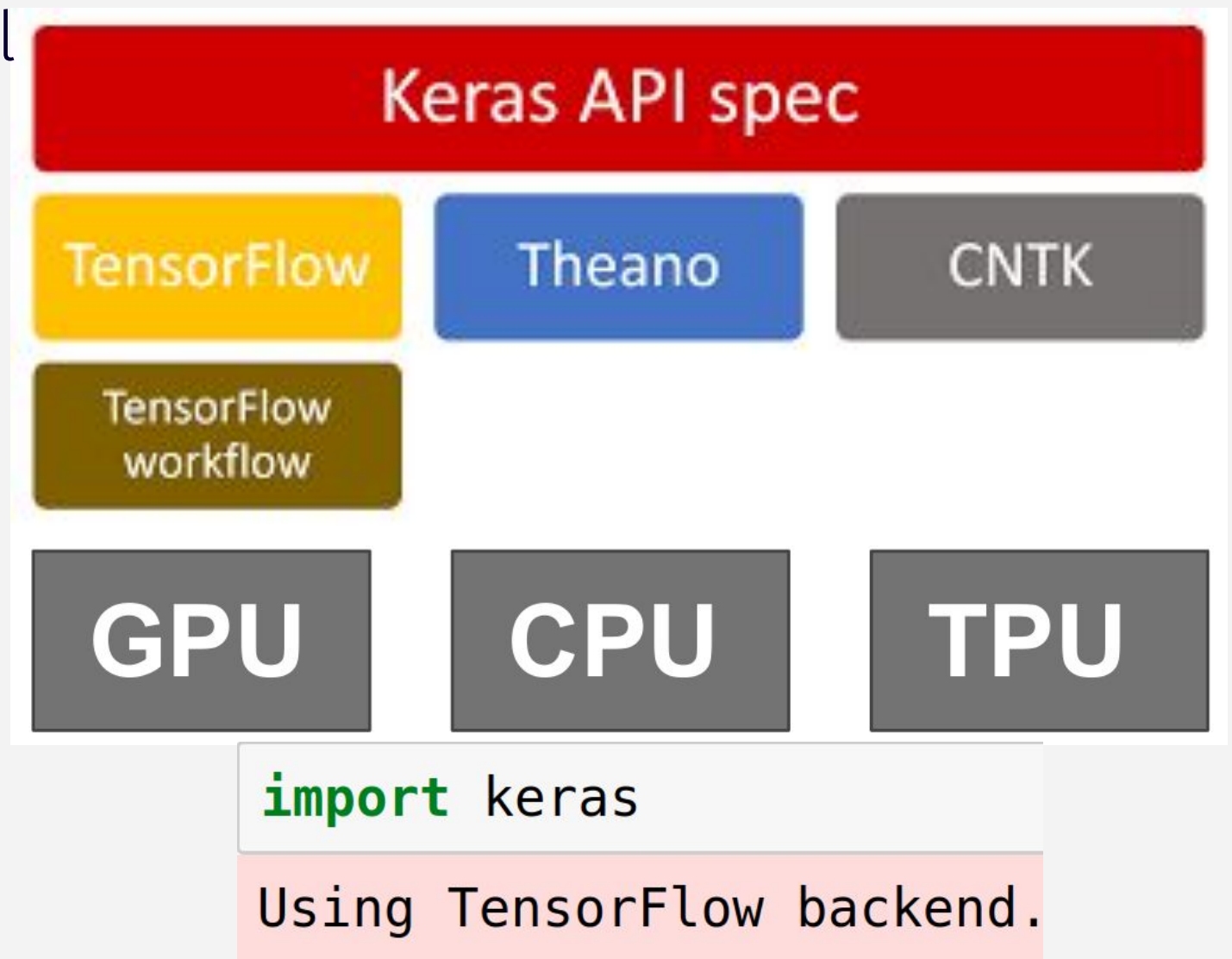
Tensorflow - Arquitectura

- Core en C++
- API en varios lenguajes
- Elementos
 - API de operadores
 - Grafo de computación
 - **Derivadas automáticas**
 - Implementación optimizada para cada dispositivo
 - Comunicación entre dispositivos



Keras - Arquitectura

- Keras: interfaz de alto nivel
 - Otras libs hacen el trabajo pesado (*backends*)
- Para definir Redes Neuronales
 - Y otros modelos diferenciables
- Provee API unificada para:
 - Cargar datos
 - Entrenar (fit)
 - Predecir (predict)
 - Evaluar (evaluate)



TensorFlow - Nueva API Eager

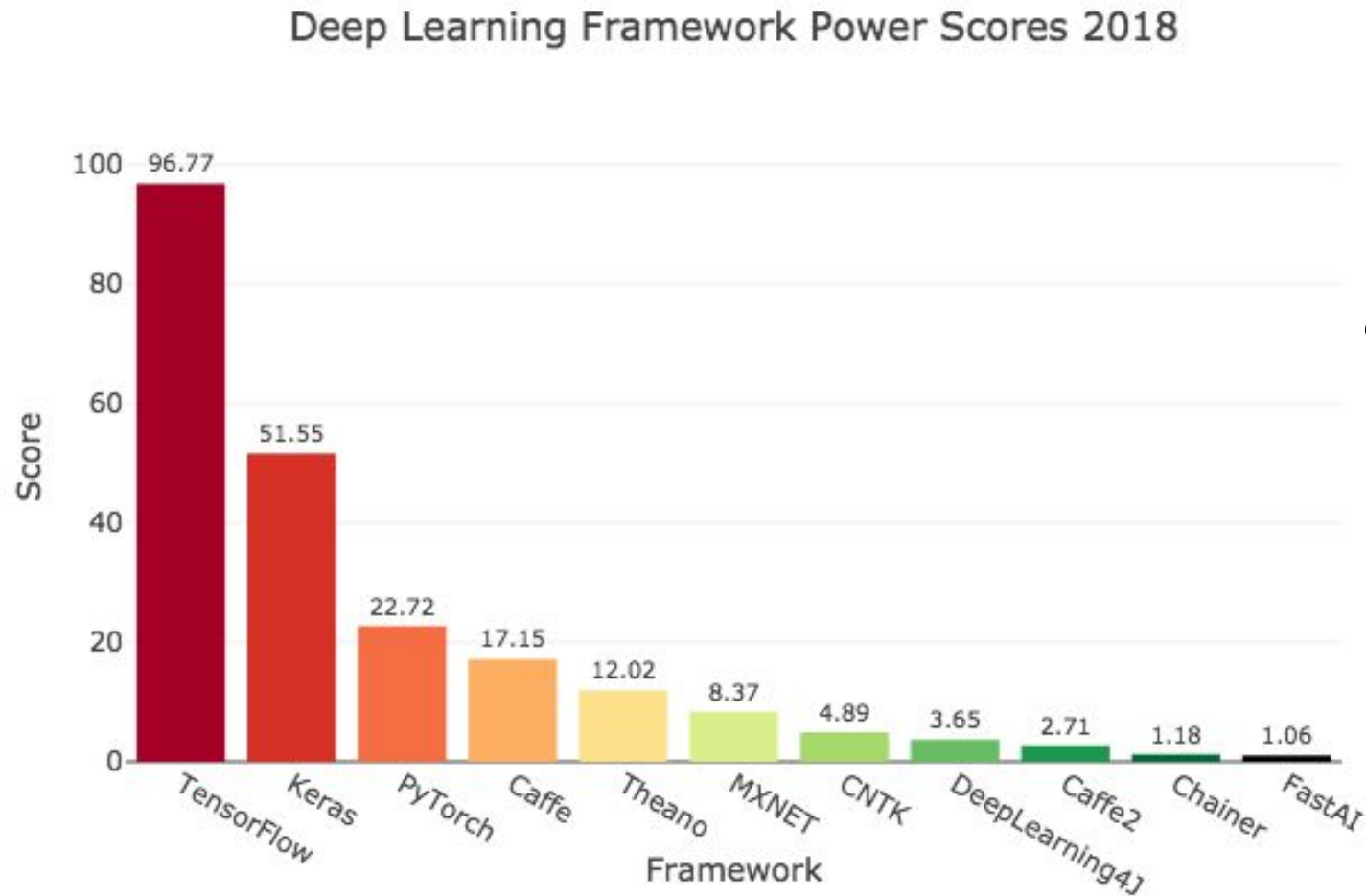
- Nueva API permite ejecución **Eager**
 - Imperativa
 - Se ejecutan inmediatamente
 - Keras NO las utiliza
- Ejemplo con API eager

```
# habilitar API eager
tf.enable_eager_execution()

x = INPUT # un vector de numpy
W = tf.Variable(np.random.randn())
b = tf.Variable(np.random.randn())

# Salida
C = tf.nn.relu(tf.add(tf.multiply(x, W), b))
# C ya tiene el output (vector de numpy)
```

TensorFlow vs Keras vs Otros (2018)



- En este curso sólo usamos Keras
 - Simple
 - Fácil de aprender
- El interop entre tf y keras es muy bueno
 - Podés combinar ambos en un mismo proyecto
 - Usar tf solo donde es necesario

Resumen

Keras

- Alto nivel (capas)
- Usa TF como backend
 - Backend: provee la implementación real de las operaciones
- Fácil uso
- Input y output con arreglos de numpy
- Puede usar operadores/capas custom implementadas en TF

Tensorflow (TF)

- Nivel intermedio (tensores)
 - tensores = matrices n-dim
- Muy customizable
- Útil para definir nuevos operadores
- tf.tensor: clase propia de tensor
 - convertir desde/hacia numpy
- Repetitivo para redes
 - Keras está incluido en TF para facilitar su uso
- Entrenamiento y predicción “manuales”